University of Asia Pacific

Department of Computer Science and Engineering

Course Code : CSE 450 (VLSI Lab)

Final Project | Spring 2026

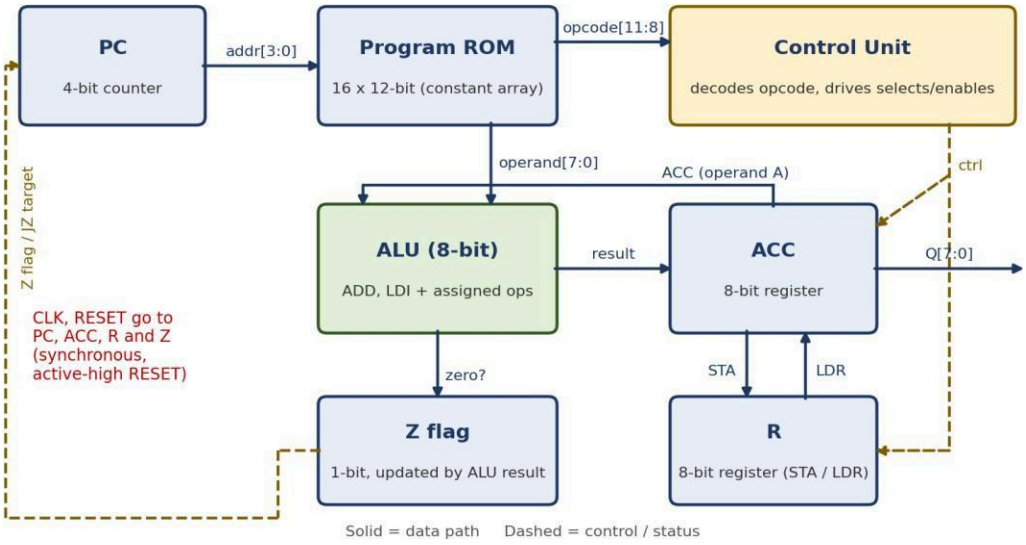
Total Marks	50 (Viva → 30, Report & VrC → 10, Presentation → 10)		
Student Name	Md Istiak Ahammed Nabil	**ID**	22201133

Final Project: Mini 8-bit Accumulator CPU

Objective. Design and implement a small single-cycle 8-bit accumulator CPU in VHDL. The CPU supports only five instructions: two are mandatory for every student, and the remaining three are assigned to you according to your Student ID (see the “Instruction Set Assignment” page: the last digit of your Student ID decides your set). You must also verify the design with a testbench that runs the program given for your set in Appendix B.

1. System Architecture

The CPU is built from the blocks shown below. Every sequential block (PC, ACC, R, Z) is clocked on the rising edge of CLK and cleared by a synchronous, active-high RESET.



Block	Description
PC	4-bit program counter. Holds the address of the instruction being executed. After every instruction it becomes PC + 1, except when JZ is taken.
Program ROM	16 locations x 12 bits, written as a constant array in VHDL. It is read combinationally using PC (no clock needed). Unused locations contain 0x000 (NOP).
Control Unit	Decodes opcode[11:8] and generates the ALU function select and the load/enable signals for ACC, R, Z and PC.
ALU	8-bit combinational unit working on ACC and the 8-bit operand. Only the operations of your own instruction set are required.
ACC, R, Z	ACC: 8-bit accumulator, also the CPU output Q[7:0]. R: 8-bit holding register (only if your set has STA / LDR). Z: 1-bit flag, 1 when the value written to ACC is 0x00.

2. Instruction Format

Each instruction is 12 bits: **instr[11:8] = opcode** (4 bits) and **instr[7:0] = operand** (8-bit immediate). Every instruction takes exactly one clock cycle (fetch, decode and execute happen in the same cycle, the result is written at the rising edge).

Mandatory instructions (all students):

Mnemonic	Opcode	Operation	Flags
LDI imm	0001	ACC ← imm	Z updated
ADD imm	0010	ACC ← ACC + imm (8-bit, carry discarded)	Z updated

Instruction pool (you implement only the three assigned to you):

Mnemonic	Opcode	Operation	Flags
SUB imm	0011	ACC ← ACC − imm (modulo 256)	Z updated
AND imm	0100	ACC ← ACC AND imm	Z updated
OR imm	0101	ACC ← ACC OR imm	Z updated
XOR imm	0110	ACC ← ACC XOR imm	Z updated
NOT	0111	ACC ← NOT ACC (operand ignored)	Z updated
SHL	1000	ACC ← ACC shifted left by 1, 0 enters LSB (operand ignored)	Z updated
SHR	1001	ACC ← ACC shifted right by 1, 0 enters MSB (operand ignored)	Z updated
STA	1010	R ← ACC (operand ignored, ACC unchanged)	Z unchanged
LDR	1011	ACC ← R (operand ignored)	Z updated
JZ addr	1100	If Z = 1 then PC ← operand[3:0], else PC ← PC + 1	Z unchanged

Note: if your set contains LDR it also contains STA. Any opcode not in your set (including 0000) behaves as NOP: PC ← PC + 1 and nothing else changes.

3. Reset Behaviour

RESET = 1 at a rising edge: PC ← 0, ACC ← 0x00, R ← 0x00, Z ← 1 (RESET has priority over everything). **RESET = 0 at a rising edge:** execute the instruction at ROM[PC]. Between clock edges all registers keep their values.

4. Implementation Requirements

- Use VHDL with IEEE.STD_LOGIC_1164.ALL (IEEE.NUMERIC_STD is allowed for arithmetic).
- Modular design: separate entities for PC, Program ROM, Control Unit, ALU and ACC/R/Z, connected in a top-level CPU entity.
- RESET must be synchronous and active-high. All state changes must occur on the rising edge of CLK.
- The top level must expose CLK, RESET, Q[7:0] (the ACC), the Z flag and, for checking, PC[3:0].
- Implement only the five instructions of your own set. The ROM must hold exactly the Appendix B program of your set; the testbench must generate a clock, apply RESET for one rising edge, and run until the last instruction has executed.

Instruction Set Assignment

How to find your set: take the **last digit of your Student ID** (ID mod 10). That digit is your set number in the table below. Example: ID 22201014 → last digit 4 → Set 4.

Every student implements the two mandatory instructions **LDI** and **ADD** plus the three assigned instructions of his/her set. Total = 5 instructions.

Last digit of ID	Set	Mandatory	Assigned	Complete instruction set of the CPU
0	**Set 0**	LDI, ADD	AND, JZ, NOT	LDI, ADD, AND, JZ, NOT
1	**Set 1**	LDI, ADD	SUB, AND, OR	LDI, ADD, SUB, AND, OR
2	**Set 2**	LDI, ADD	XOR, NOT, SHL	LDI, ADD, XOR, NOT, SHL
3	**Set 3**	LDI, ADD	SHR, SUB, XOR	LDI, ADD, SHR, SUB, XOR
4	**Set 4**	LDI, ADD	STA, LDR, AND	LDI, ADD, STA, LDR, AND

Last digit of ID	Set	Mandatory	Assigned	Complete instruction set of the CPU
5	**Set 5**	LDI, ADD	OR, NOT, SHR	LDI, ADD, OR, NOT, SHR
6	**Set 6**	LDI, ADD	SUB, JZ, SHL	LDI, ADD, SUB, JZ, SHL
7	**Set 7**	LDI, ADD	AND, XOR, SHR	LDI, ADD, AND, XOR, SHR
8	**Set 8**	LDI, ADD	STA, LDR, SUB	LDI, ADD, STA, LDR, SUB
9	**Set 9**	LDI, ADD	NOT, SHL, JZ	LDI, ADD, NOT, SHL, JZ

Appendix B: Required Test Program and Expected Results

Store the program of your set (Appendix B tables are ordered by set number 0–9) in the Program ROM. Apply RESET for one rising edge, then run the CPU for one clock cycle per executed instruction. The ACC, Z, R and next-PC columns show the values after the corresponding rising edge. Addresses that are not listed contain 0x000 (NOP). Operands are in hexadecimal. For NOT, SHL, SHR, STA and LDR the operand field is 0x00.

ROM encoding: instr[11:0] = opcode (4 bit) & operand (8 bit). Example: LDI 0x3C = 0001 00111100 = 0x13C.

Set 0: LDI, ADD, AND, JZ, NOT

ROM contents: 0: LDI 0xA5 → 0x1A5 | 1: AND 0x5A → 0x45A | 2: JZ 0x05 → 0xC05 | 3: ADD 0x01 → 0x201 | 4: ADD 0x01 → 0x201 | 5: NOT → 0x700 | 6: AND 0x0F → 0x40F | 7: JZ 0x00 → 0xC00 | 8: ADD 0x01 → 0x201

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0xA5	0xA5	0	10100101	1
2	1	AND 0x5A	0x00	1	00000000	2
3	2	JZ 0x05 (taken)	0x00	1	00000000	5
4	5	NOT	0xFF	0	11111111	6
5	6	AND 0x0F	0x0F	0	00001111	7
6	7	JZ 0x00 (not taken)	0x0F	0	00001111	8
7	8	ADD 0x01	0x10	0	00010000	9

Set 1: LDI, ADD, SUB, AND, OR

ROM contents: 0: LDI 0x3C → 0x13C | 1: ADD 0x0F → 0x20F | 2: SUB 0x0B → 0x30B | 3: AND 0xF0 → 0x4F0 | 4: OR 0x0A → 0x50A | 5: SUB 0x4A → 0x34A | 6: SUB 0x01 → 0x301

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x3C	0x3C	0	00111100	1
2	1	ADD 0x0F	0x4B	0	01001011	2
3	2	SUB 0x0B	0x40	0	01000000	3
4	3	AND 0xF0	0x40	0	01000000	4
5	4	OR 0x0A	0x4A	0	01001010	5
6	5	SUB 0x4A	0x00	1	00000000	6
7	6	SUB 0x01	0xFF	0	11111111	7

Set 2: LDI, ADD, XOR, NOT, SHL

ROM contents: 0: LDI 0x55 → 0x155 | 1: XOR 0xFF → 0x6FF | 2: NOT → 0x700 | 3: SHL → 0x800 | 4: ADD 0x56 → 0x256 | 5: XOR 0x81 → 0x681 | 6: SHL → 0x800

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x55	0x55	0	01010101	1
2	1	XOR 0xFF	0xAA	0	10101010	2
3	2	NOT	0x55	0	01010101	3
4	3	SHL	0xAA	0	10101010	4
5	4	ADD 0x56	0x00	1	00000000	5
6	5	XOR 0x81	0x81	0	10000001	6
7	6	SHL	0x02	0	00000010	7

Set 3: LDI, ADD, SHR, SUB, XOR

ROM contents: 0: LDI 0x80 → 0x180 | 1: SHR → 0x900 | 2: SUB 0x10 → 0x310 | 3: ADD 0x0F → 0x20F | 4: XOR 0x3F → 0x63F | 5: ADD 0x07 → 0x207 | 6: SHR → 0x900

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x80	0x80	0	10000000	1
2	1	SHR	0x40	0	01000000	2
3	2	SUB 0x10	0x30	0	00110000	3
4	3	ADD 0x0F	0x3F	0	00111111	4
5	4	XOR 0x3F	0x00	1	00000000	5
6	5	ADD 0x07	0x07	0	00000111	6
7	6	SHR	0x03	0	00000011	7

Set 4: LDI, ADD, STA, LDR, AND

ROM contents: 0: LDI 0x1E → 0x11E | 1: STA → 0xA00 | 2: ADD 0x22 → 0x222 | 3: AND 0x60 → 0x460 | 4: LDR → 0xB00 | 5: ADD 0x01 → 0x201 | 6: AND 0x0F → 0x40F | 7: STA → 0xA00 | 8: LDI 0x00 → 0x100 | 9: LDR → 0xB00

Cycle	PC	Instruction executed	ACC (hex)	Z	R (hex)	ACC (bin)	Next PC
0	-	RESET = 1	0x00	1	0x00	00000000	0
1	0	LDI 0x1E	0x1E	0	0x00	00011110	1
2	1	STA	0x1E	0	0x1E	00011110	2
3	2	ADD 0x22	0x40	0	0x1E	01000000	3
4	3	AND 0x60	0x40	0	0x1E	01000000	4
5	4	LDR	0x1E	0	0x1E	00011110	5
6	5	ADD 0x01	0x1F	0	0x1E	00011111	6
7	6	AND 0x0F	0x0F	0	0x1E	00001111	7
8	7	STA	0x0F	0	0x0F	00001111	8
9	8	LDI 0x00	0x00	1	0x0F	00000000	9
10	9	LDR	0x0F	0	0x0F	00001111	10

Set 5: LDI, ADD, OR, NOT, SHR

ROM contents: 0: LDI 0x0F → 0x10F | 1: OR 0xA0 → 0x5A0 | 2: NOT → 0x700 | 3: SHR → 0x900 | 4: ADD 0xD8 → 0x2D8 | 5: OR 0x81 → 0x581 | 6: SHR → 0x900

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x0F	0x0F	0	00001111	1
2	1	OR 0xA0	0xAF	0	10101111	2
3	2	NOT	0x50	0	01010000	3
4	3	SHR	0x28	0	00101000	4
5	4	ADD 0xD8	0x00	1	00000000	5
6	5	OR 0x81	0x81	0	10000001	6
7	6	SHR	0x40	0	01000000	7

Set 6: LDI, ADD, SUB, JZ, SHL

ROM contents: 0: LDI 0x02 → 0x102 | 1: SUB 0x01 → 0x301 | 2: JZ 0x07 → 0xC07 | 3: SUB 0x01 → 0x301 | 4: JZ 0x07 → 0xC07 | 5: ADD 0xAA → 0x2AA | 6: ADD 0xAA → 0x2AA | 7: LDI 0x21 → 0x121 | 8: SHL → 0x800 | 9: ADD 0x01 → 0x201

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x02	0x02	0	00000010	1
2	1	SUB 0x01	0x01	0	00000001	2
3	2	JZ 0x07 (not taken)	0x01	0	00000001	3
4	3	SUB 0x01	0x00	1	00000000	4
5	4	JZ 0x07 (taken)	0x00	1	00000000	7
6	7	LDI 0x21	0x21	0	00100001	8
7	8	SHL	0x42	0	01000010	9
8	9	ADD 0x01	0x43	0	01000011	10

Set 7: LDI, ADD, AND, XOR, SHR

ROM contents: 0: LDI 0xF3 → 0x1F3 | 1: AND 0x3C → 0x43C | 2: XOR 0x11 → 0x611 | 3: SHR → 0x900 | 4: ADD 0x30 → 0x230 | 5: XOR 0x40 → 0x640

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0xF3	0xF3	0	11110011	1
2	1	AND 0x3C	0x30	0	00110000	2
3	2	XOR 0x11	0x21	0	00100001	3
4	3	SHR	0x10	0	00010000	4
5	4	ADD 0x30	0x40	0	01000000	5
6	5	XOR 0x40	0x00	1	00000000	6

Set 8: LDI, ADD, STA, LDR, SUB

ROM contents: 0: LDI 0x64 → 0x164 | 1: STA → 0xA00 | 2: SUB 0x14 → 0x314 | 3: ADD 0x0A → 0x20A | 4: STA → 0xA00 | 5: SUB 0x5A → 0x35A | 6: LDR → 0xB00 | 7: SUB 0x60 → 0x360

Cycle	PC	Instruction executed	ACC (hex)	Z	R (hex)	ACC (bin)	Next PC
0	-	RESET = 1	0x00	1	0x00	00000000	0
1	0	LDI 0x64	0x64	0	0x00	01100100	1
2	1	STA	0x64	0	0x64	01100100	2
3	2	SUB 0x14	0x50	0	0x64	01010000	3
4	3	ADD 0x0A	0x5A	0	0x64	01011010	4
5	4	STA	0x5A	0	0x5A	01011010	5
6	5	SUB 0x5A	0x00	1	0x5A	00000000	6
7	6	LDR	0x5A	0	0x5A	01011010	7
8	7	SUB 0x60	0xFA	0	0x5A	11111010	8

Set 9: LDI, ADD, NOT, SHL, JZ

ROM contents: 0: LDI 0xFF → 0x1FF | 1: NOT → 0x700 | 2: JZ 0x05 → 0xC05 | 3: ADD 0x77 → 0x277 | 4: ADD 0x77 → 0x277 | 5: ADD 0x11 → 0x211 | 6: SHL → 0x800 | 7: JZ 0x00 → 0xC00 | 8: SHL → 0x800

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0xFF	0xFF	0	11111111	1
2	1	NOT	0x00	1	00000000	2
3	2	JZ 0x05 (taken)	0x00	1	00000000	5
4	5	ADD 0x11	0x11	0	00010001	6
5	6	SHL	0x22	0	00100010	7
6	7	JZ 0x00 (not taken)	0x22	0	00100010	8
7	8	SHL	0x44	0	01000100	9

Submission Guidelines

Submit your VHDL source files, testbench and a project report within the 2-week period. Include this project description (question, assignment page and Appendix B) at the front of your report and start your own report on a new page after it. The report must include, but is not limited to, the following points.

- Your Student ID, assigned set number and the three assigned instructions (highlighted in yellow color)
- Attach your entity code (all modules), testbench code and output visuals (waveform with PC, ACC, Z and R if used)
- Challenges faced during the experiment, and conclusion

Projcct Rc2ort: Mini 8-bit Accumulator CPU in VHDL

CSE 450 (VLSI Lab) · Final Project · Spring 2026 · Md Istiak Ahammed Nabil (22201133, Section C)

Oct 23, 2026.

Projcct Dcscri2tion

The project question, the instruction set assignment and Appendix B (5 pages, with name and ID filled in) are attached at the front of the submitted PDF, as the submission guidelines require. The report itself starts at Section 1.

1. Studcnt Infiormation and Assigncd Instruction Sct

Item	Details
Student Name	Md Istiak Ahammed Nabil
Student ID	22201133 (last digit = 3)
Section	C
Assigned set	**Set 3**
Mandatory instructions	LDI, ADD
Assigned instructions	**SHR, SUB, XOR**
Complete instruction set	LDI, ADD, **SHR**, **SUB**, **XOR**
Language / tools	VHDL (IEEE.STD_LOGIC_1164), Xilinx ISE 14.7 project (XST synthesis, ISim simulation), target device Artix-7 xc7a100t. Waveforms in this report are from the testbench runs of the final code.

The CPU implements five single-cycle instructions with a 12-bit format: opcode[11:8] and operand[7:0]. The three instructions assigned to Set 3 are highlighted in Table 1.

Table 1: Instruction set implemented by this CPU (Set 3)

Mnemonic	Opcode	Operation	ALU_OP	Flags
LDI imm	0001	ACC <- imm	(operand bypasses ALU)	Z updated
ADD imm	0010	ACC <- ACC + imm (8-bit, carry discarded)	00	Z updated
SUB imm	0011	ACC <- ACC - imm (modulo 256)	10	Z updated
XOR imm	0110	ACC <- ACC XOR imm	01	Z updated
SHR	1001	ACC <- ACC shifted right by 1, 0 enters MSB (operand ignored)	11	Z updated

Every other opcode, including 0000, is a NOP: the accumulator keeps its value and PC <-PC + 1.

2. Dcsign Ovcrvicw

The processor was designed bottom-up. Every logic function comes from a single 2-input NAND gate. The gates are combined into adders, a multiplexer, latches and flip-flops, then into an 8-bit ALU, a zero detector and registers, and finally into the complete CPU. Most modules have their own testbench, and the top-level testbench runs the Set 3 program of Appendix B and checks PC, ACC and Z after every clock edge with assertions.

2.1 Modulc hicrarchy

Table 2: Module hierarchy (lowest level first)

Level	Module	Built from	Role
0	nand_gate	Dataflow: Y <= A nand B	Only primitive gate
1	not_gate, and_gate, or_gate, xor_gate	NAND gates only	Basic logic

2	half_adder, full_adder, mux_8bit, d_latch	XOR / AND / OR / NOT / NAND gates	Arithmetic, selection, 1-bit storage

Level	Module	Built from	Role
3	full_adder_8bit, d_flip_flop	8 full adders (ripple carry); two D latches (master-slave)	8-bit adder, edge-triggered bit
4	register_8bit	8 D flip-flops + mux_8bit (reset to zero)	8-bit register with synchronous reset
5	alu_8bit, zero_flag, accumulator, program_counter	Level 1 to 4 modules	Datapath blocks
5	program_rom, instruction_decoder, control_unit	Behavioural (case statements, slices)	Program storage and control
6	cpu_top	All of the above	Complete CPU

2.2 CPU architecture

Figure 1 shows how the blocks of the final design are connected. The Program Counter addresses the Program ROM, and the 12-bit instruction goes to the Instruction Decoder, which splits it into OPCODE and OPERAND. The Control Unit turns the opcode into three signals: LOAD_EN (LDI), ACC_WR (write the accumulator) and the 2-bit ALU_OP. The ALU computes ACC op OPERAND. The ACC next-value logic picks the operand for LDI, the ALU result for ADD, SUB, XOR and SHR, or keeps the old value for a NOP. The Accumulator stores it on the rising clock edge, and the Zero Flag watches ACC. Set 3 has no jump, so the PC simply adds 1 on every edge.

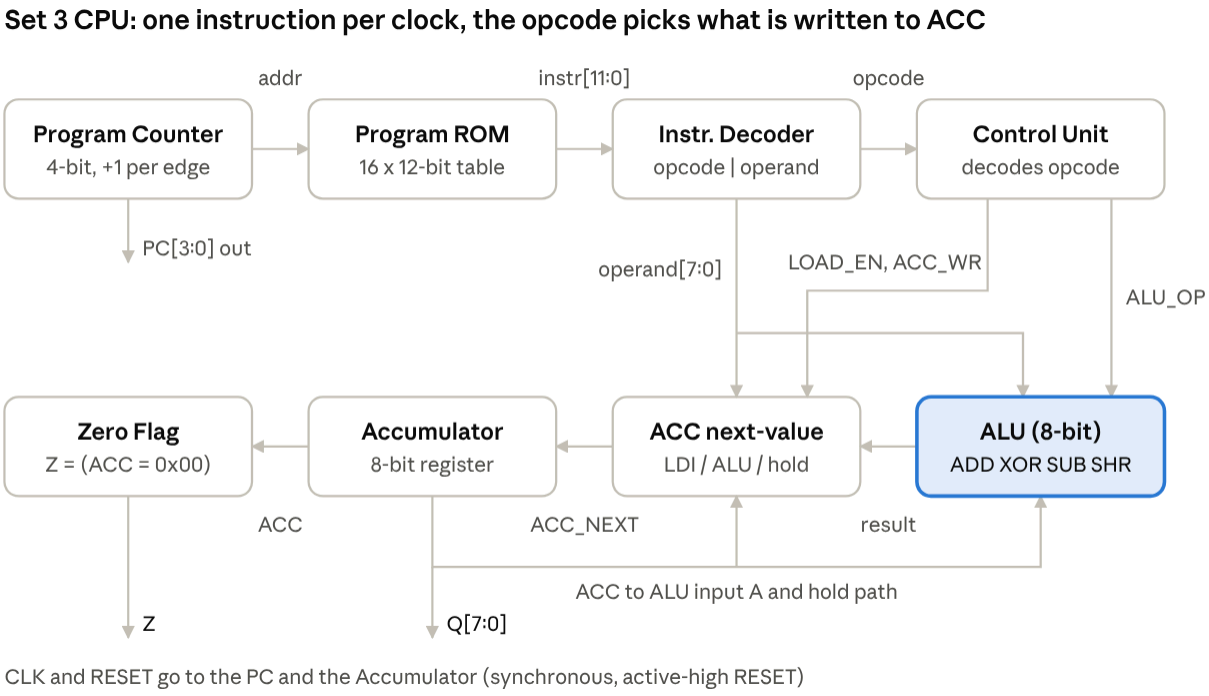


Figure 1: block diagram of the implemented 8-bit accumulator CPU (Set 3)

Figure 1: Block diagram of the implemented 8-bit accumulator CPU (Set 3)

2.3 Program storcd in thc ROM

The Program ROM holds exactly the Appendix B program of Set 3. Unused locations return 0x000, which is decoded as NOP.

Table 3: Program ROM contents (Set 3)

Address	ROM word	Instruction	Encoding (opcode & operand)
0	0x180	LDI 0x80	0001 & 10000000
1	0x900	**SHR**	1001 & 00000000
2	0x310	**SUB 0x10**	0011 & 00010000
3	0x20F	ADD 0x0F	0010 & 00001111
4	0x63F	**XOR 0x3F**	0110 & 00111111
5	0x207	ADD 0x07	0010 & 00000111
6	0x900	**SHR**	1001 & 00000000
7 to 15	0x000	NOP	0000 & 00000000

3. Basic Building Blocks

These small blocks are the foundation of the datapath. They were verified by their own testbenches (the D latch and D flip-flop have no separate testbench; they are exercised inside the register and the full CPU), so only their code is shown here. Repeated component declarations are omitted to save space.

3.1 NAND gatc

The only primitive of the design. All other gates are made from it.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity nand_gate is
    Port ( A : in STD_LOGIC; B : in STD_LOGIC; Y : out STD_LOGIC );
end nand_gate;
architecture Dataflow of nand_gate is
begin
    Y <= A nand B;
end Dataflow;
```

3.2 NOT gatc

A NAND gate with both inputs tied together: Y = NOT(A).

```
entity not_gate is
    Port ( A : in STD_LOGIC; Y : out STD_LOGIC );
end not_gate;
architecture Structural of not_gate is
    -- component declarations omitted: nand_gate
begin
    NAND1: nand_gate port map (A => A, B => A, Y => Y);
end Structural;
```

3.3 AND gatc

A NAND followed by a second NAND used as an inverter.

```
entity and_gate is
    Port ( A : in STD_LOGIC; B : in STD_LOGIC; Y : out STD_LOGIC );
end and_gate;
architecture Structural of and_gate is
    -- component declarations omitted: nand_gate
    signal nand_out : STD_LOGIC;
begin
    NAND1: nand_gate port map (A => A, B => B, Y => nand_out);
    NAND2: nand_gate port map (A => nand_out, B => nand_out, Y => Y);
end Structural;
```

3.4 OR gatc

Two inverted inputs into a NAND: Y = A OR B (De Morgan).

```
entity or_gate is
    Port ( A : in STD_LOGIC; B : in STD_LOGIC; Y : out STD_LOGIC );
end or_gate;
architecture Structural of or_gate is
    -- component declarations omitted: nand_gate
    signal not_A : STD_LOGIC;
    signal not_B : STD_LOGIC;
begin
    NAND1: nand_gate port map (A => A, B => A, Y => not_A);
    NAND2: nand_gate port map (A => B, B => B, Y => not_B);
    NAND3: nand_gate port map (A => not_A, B => not_B, Y => Y);
end Structural;
```

3.5 XOR gatc

The standard four-NAND XOR structure. It is used in the half adder and directly in the ALU for the XOR instruction.

```
entity xor_gate is
    Port ( A : in STD_LOGIC; B : in STD_LOGIC; Y : out STD_LOGIC );
end xor_gate;
architecture Structural of xor_gate is
    -- component declarations omitted: nand_gate
    signal nand1_out, nand2_out, nand3_out : STD_LOGIC;
begin
    NAND1: nand_gate port map (A => A, B => B,    Y => nand1_out);
    NAND2: nand_gate port map (A => A, B => nand1_out, Y => nand2_out);
    NAND3: nand_gate port map (A => B, B => nand1_out, Y => nand3_out);
    NAND4: nand_gate port map (A => nand2_out, B => nand3_out, Y => Y);
end Structural;
```

3.6 Halfi addcr

SUM = A XOR B and CARRY = A AND B.

```
entity half_adder is
    Port ( A, B : in STD_LOGIC; SUM : out STD_LOGIC; CARRY : out STD_LOGIC );
end half_adder;
architecture Structural of half_adder is
    -- component declarations omitted: xor_gate, and_gate
begin
    XOR1: xor_gate port map (A => A, B => B, Y => SUM);
    AND1: and_gate port map (A => A, B => B, Y => CARRY);
end Structural;
```

3.7 Full addcr

Two half adders and an OR gate for the carry.

```
entity full_adder is
    Port ( A, B, CIN : in STD_LOGIC; SUM, COUT : out STD_LOGIC );
end full_adder;
architecture Structural of full_adder is
    -- component declarations omitted: half_adder, or_gate
    signal sum1, carry1, carry2 : STD_LOGIC;
begin
    HA1: half_adder port map (A => A,    B => B,   SUM => sum1, CARRY =>
carry1);
    HA2: half_adder port map (A => sum1, B => CIN, SUM => SUM, CARRY =>
carry2);
    OR1: or_gate port map (A => carry1, B => carry2, Y => COUT);
end Structural;
```

3.8 8-bit ri22lc-carry addcr

Eight full adders chained through the carry vector C. It is the core of the ALU (used twice: once for ADD and once for SUB) and of the program counter's +1 incrementer. FA2 to FA6 follow the same pattern and are omitted here.

```
entity full_adder_8bit is
    Port (
        A    : in STD_LOGIC_VECTOR(7 downto 0);
        B     : in STD_LOGIC_VECTOR(7 downto 0);
        CIN : in STD_LOGIC;
        SUM : out STD_LOGIC_VECTOR(7 downto 0);
        COUT : out STD_LOGIC
    );
end full_adder_8bit;
architecture Structural of full_adder_8bit is
    -- component declarations omitted: full_adder
    signal C : STD_LOGIC_VECTOR(8 downto 0);
begin
    C(0) <= CIN;
    FA0: full_adder port map (A => A(0), B => B(0), CIN => C(0), SUM =>
SUM(0), COUT => C(1));
    FA1: full_adder port map (A => A(1), B => B(1), CIN => C(1), SUM =>
SUM(1), COUT => C(2));
    -- FA2 ... FA6 : same pattern (A(i), B(i), C(i) -> SUM(i), C(i+1))
```

```
    FA7: full_adder port map (A => A(7), B => B(7), CIN => C(7), SUM =>
SUM(7), COUT => C(8));
    COUT <= C(8);
end Structural;
```

3.9 8-bit 2-to-1 multi2lcxcr

Gate-level multiplexer: SEL = 0 selects A, SEL = 1 selects B. Inside the register it selects zero while RESET = 1, which is how the synchronous reset is built.

```
entity mux_8bit is
    Port (
        A, B : in STD_LOGIC_VECTOR(7 downto 0);
        SEL : in STD_LOGIC;
        Y    : out STD_LOGIC_VECTOR(7 downto 0)
    );
end mux_8bit;
architecture Structural of mux_8bit is
    -- component declarations omitted: not_gate, and_gate, or_gate
    signal NOT_SEL : STD_LOGIC;
    signal AND_A, AND_B : STD_LOGIC_VECTOR(7 downto 0);
begin
    INV1: not_gate port map (A => SEL, Y => NOT_SEL);
    GEN_MUX: for i in 0 to 7 generate
        AND1: and_gate port map (A => A(i),     B => NOT_SEL, Y =>
AND_A(i));
        AND2: and_gate port map (A => B(i),     B => SEL,      Y =>
AND_B(i));
        OR1: or_gate port map (A => AND_A(i), B => AND_B(i), Y => Y(i));
    end generate GEN_MUX;
end Structural;
```

3.10 D latch

A gated D latch made of NAND gates: two input-gating NANDs and a cross-coupled NAND pair. While EN = 1, Q follows D; while EN = 0, Q holds.

```
entity d_latch is
    Port ( D : in STD_LOGIC; EN : in STD_LOGIC; Q : out STD_LOGIC );
end d_latch;
architecture Structural of d_latch is
    -- component declarations omitted: nand_gate, not_gate
    signal D_NOT, S_N, R_N, Q_INT, QN_INT : STD_LOGIC;
begin
    -- Invert D
    INV_D   : not_gate port map (A => D, Y => D_NOT);
    -- NAND-based input gating
    NAND_S  : nand_gate port map (A => D,     B => EN,     Y => S_N);
    NAND_R  : nand_gate port map (A => D_NOT, B => EN,     Y => R_N);
    -- Cross-coupled NAND latch
    NAND_Q : nand_gate port map (A => S_N,  B => QN_INT, Y => Q_INT);
    NAND_QN : nand_gate port map (A => R_N,   B => Q_INT,  Y => QN_INT);
    Q <= Q_INT;
end Structural;
```

3.11 D fili2-filo2

A master-slave flip-flop from two D latches. The master is open while CLK = 0 and the slave while CLK = 1, so Q changes only on the rising edge.

```
entity d_flip_flop is
    Port ( D, CLK, RESET : in STD_LOGIC; Q : out STD_LOGIC );
end d_flip_flop;
architecture Structural of d_flip_flop is
    -- component declarations omitted: d_latch
    signal CLK_NOT, MASTER_Q, SLAVE_Q : STD_LOGIC;
begin
    -- Invert clock
    CLK_INV : entity work.not_gate port map (A => CLK, Y => CLK_NOT);
    -- Master latch enabled when CLK = 0
    MASTER_LATCH : d_latch port map (D => D, EN => CLK_NOT, Q => MASTER_Q);
    -- Slave latch enabled when CLK = 1
    SLAVE_LATCH : d_latch port map (D => MASTER_Q, EN => CLK, Q => SLAVE_Q);
    -- Synchronous reset
    process(CLK)
    begin
        if rising_edge(CLK) then
            if RESET = '1' then
                -- Reset is handled at the output below
```

```
                null;
            end if;
        end if;
    end process;
    Q <= SLAVE_Q when RESET = '0' else '0';
end Structural;
```

3.12 8-bit register

Eight D flip-flops with a synchronous, active-high reset on the rising clock edge. The reset goes through the mux_8bit, which feeds zero into the flip-flops while RESET = 1. It is used by the program counter and the accumulator.

```
entity register_8bit is
    Port (
        D     : in STD_LOGIC_VECTOR(7 downto 0);
        CLK   : in STD_LOGIC;
        RESET : in STD_LOGIC;
        Q     : out STD_LOGIC_VECTOR(7 downto 0)
    );
end register_8bit;
architecture Structural of register_8bit is
    signal REG_DATA : STD_LOGIC_VECTOR(7 downto 0);
    signal REG_NEXT : STD_LOGIC_VECTOR(7 downto 0);
    signal CLK_NOT : STD_LOGIC;
begin
    -- Invert the clock using the NOT gate
    CLK_INV : entity work.not_gate port map (A => CLK, Y => CLK_NOT);
    -- Feedback mux: select zero during reset, otherwise D
    REG_MUX : entity work.mux_8bit
        port map (
            A   => D,                   -- SEL = '0' (normal): pass D
            B   => (others => '0'),     -- SEL = '1' (RESET): zero
            SEL => RESET,
            Y   => REG_NEXT
        );
    -- Eight storage bits
    REG_BITS : for i in 0 to 7 generate
        BIT_REG : entity work.d_flip_flop
            port map (D => REG_NEXT(i), CLK => CLK, RESET => '0', Q =>
```

```
REG_DATA(i));
    end generate;
    Q <= REG_DATA;
end Structural;
```

4. Main Com2oncnts

Each main component is described below with its block symbol (XST RTL schematic), VHDL code, the stimulus used in its testbench and the simulated waveform.

4.1 ALU (alu_8bit)

The ALU is a combinational 8-bit unit with A = ACC and B = the operand. It computes all four Set 3 results in parallel and a 4-to-1 selection on the 2-bit OP picks one: 00 = ADD, 01 = XOR, 10 = SUB, 11 = SHR.

- **ADD:** one full_adder_8bit with CIN = 0. The carry-out is discarded (8-bit result).
- **XOR:** eight xor_gate instances, one per bit.
- **SUB:** a second full_adder_8bit computes A + NOT(B) + 1, the two's complement of B, so the result is A - B modulo 256.
- **SHR:** pure wiring, '0' & A(7 downto 1). A zero enters the MSB and the operand is ignored.

Figure 2: Block symbol of alu_8bit

VHDL code: alu_8bit.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity alu_8bit is
    Port (
        A      : in  STD_LOGIC_VECTOR(7 downto 0);   -- ACC
        B      : in  STD_LOGIC_VECTOR(7 downto 0);   -- operand
        OP     : in  STD_LOGIC_VECTOR(1 downto 0);   -- 00 ADD, 01 XOR, 10 SUB, 11
SHR
        RESULT : out STD_LOGIC_VECTOR(7 downto 0)
    );
end alu_8bit;
architecture Structural of alu_8bit is
    component full_adder_8bit
        Port (
            A    : in  STD_LOGIC_VECTOR(7 downto 0);
            B    : in  STD_LOGIC_VECTOR(7 downto 0);
            CIN  : in  STD_LOGIC;
            SUM  : out STD_LOGIC_VECTOR(7 downto 0);
            COUT : out STD_LOGIC
        );
    end component;
    component xor_gate
        Port ( A : in STD_LOGIC; B : in STD_LOGIC; Y : out STD_LOGIC );
    end component;
    signal ARITH_R, XOR_R, SHR_R : STD_LOGIC_VECTOR(7 downto 0);
    signal B_COND       : STD_LOGIC_VECTOR(7 downto 0);  -- B, or ~B when OP(1)='1'
    signal CARRY_UNUSED : STD_LOGIC;
begin
    GEN_LOGIC: for i in 0 to 7 generate
        -- logic XOR: A xor B
        X:  xor_gate port map (A => A(i), B => B(i),  Y => XOR_R(i));
        -- conditional inversion of B: B xor OP(1)
        XC: xor_gate port map (A => B(i), B => OP(1), Y => B_COND(i));
    end generate;
    -- One adder shared by ADD and SUB (CIN = OP(1))
    ADDSUB: full_adder_8bit
        port map (A => A, B => B_COND, CIN => OP(1), SUM => ARITH_R, COUT =>
CARRY_UNUSED);

    -- SHR : logical shift right by 1
    SHR_R <= '0' & A(7 downto 1);

    with OP select
        RESULT <= ARITH_R when "00",
                  XOR_R   when "01",
                  ARITH_R when "10",
```

```
                SHR_R   when others;
end Structural;
```

Testbench stimulus: alu_8bit_tb.vhd (excerpt)

```
-- ADD (OP = 00)
OP <= "00";
A <= x"30"; B <= x"0F"; wait for 10 ns;
assert RESULT = x"3F" report "FAIL: ADD 30 + 0F" severity error;
A <= x"00"; B <= x"07"; wait for 10 ns;
assert RESULT = x"07" report "FAIL: ADD 00 + 07" severity error;
A <= x"FF"; B <= x"01"; wait for 10 ns;
assert RESULT = x"00" report "FAIL: ADD FF + 01 (carry discarded)" severity
error;
-- XOR (OP = 01)
OP <= "01";
A <= x"3F"; B <= x"3F"; wait for 10 ns;
assert RESULT = x"00" report "FAIL: XOR 3F ^ 3F" severity error;
A <= x"00"; B <= x"81"; wait for 10 ns;
assert RESULT = x"81" report "FAIL: XOR 00 ^ 81" severity error;
A <= x"55"; B <= x"FF"; wait for 10 ns;
assert RESULT = x"AA" report "FAIL: XOR 55 ^ FF" severity error;
-- SUB (OP = 10)
OP <= "10";
A <= x"40"; B <= x"10"; wait for 10 ns;
assert RESULT = x"30" report "FAIL: SUB 40 - 10" severity error;
A <= x"55"; B <= x"55"; wait for 10 ns;
assert RESULT = x"00" report "FAIL: SUB 55 - 55" severity error;
A <= x"00"; B <= x"01"; wait for 10 ns;
assert RESULT = x"FF" report "FAIL: SUB 00 - 01 (modulo 256)" severity error;
A <= x"10"; B <= x"20"; wait for 10 ns;
assert RESULT = x"F0" report "FAIL: SUB 10 - 20 (modulo 256)" severity error;
-- SHR (OP = 11), operand ignored, 0 enters MSB
OP <= "11";
A <= x"80"; B <= x"FF"; wait for 10 ns;
assert RESULT = x"40" report "FAIL: SHR 80" severity error;
A <= x"07"; B <= x"00"; wait for 10 ns;
assert RESULT = x"03" report "FAIL: SHR 07" severity error;
A <= x"01"; B <= x"00"; wait for 10 ns;
assert RESULT = x"00" report "FAIL: SHR 01" severity error;
A <= x"FF"; B <= x"12"; wait for 10 ns;
```

```
assert RESULT = x"7F" report "FAIL: SHR FF (MSB must become 0)" severity
error;
report "alu_8bit testbench completed." severity note;
wait;
```

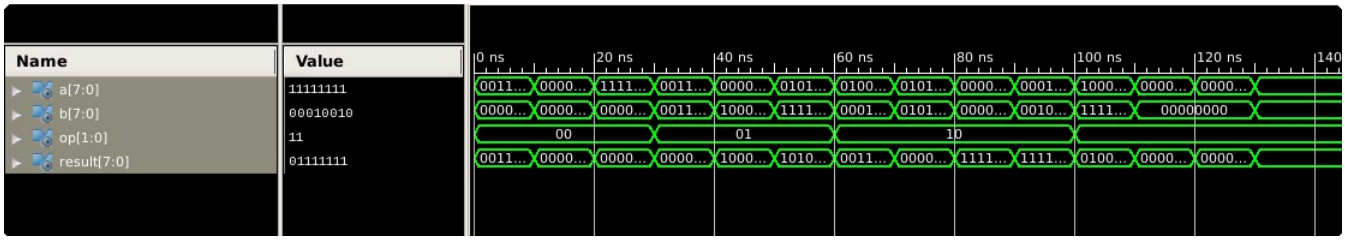


Figure 3: Simulation waveform of alu_8bit

The first three vectors use OP = 00 (ADD): 0x30 + 0x0F = 0x3F, 0x00 + 0x07 = 0x07 and 0xFF + 0x01 = 0x00 with the carry discarded. The next three use OP = 01 (XOR): 0x00, 0x81 and 0xAA. OP = 10 (SUB) gives 0x40 - 0x10 = 0x30, 0x55 - 0x55 = 0x00, 0x00 - 0x01 = 0xFF and 0x10 - 0x20 = 0xF0 (modulo 256). OP = 11 (SHR) gives 0x40, 0x03, 0x00 and 0x7F, so the MSB is always filled with 0. All results are correct.

4.2 Zcro filag (xcro_filag)

The zero detector inverts all eight bits of its input and combines them with a tree of seven AND gates, so ZERO = 1 only when every bit of A is 0. It is connected to the accumulator output, so Z always reflects the current ACC value: it is 1 after reset and is updated whenever ACC is written by LDI, ADD, SUB, XOR or SHR.

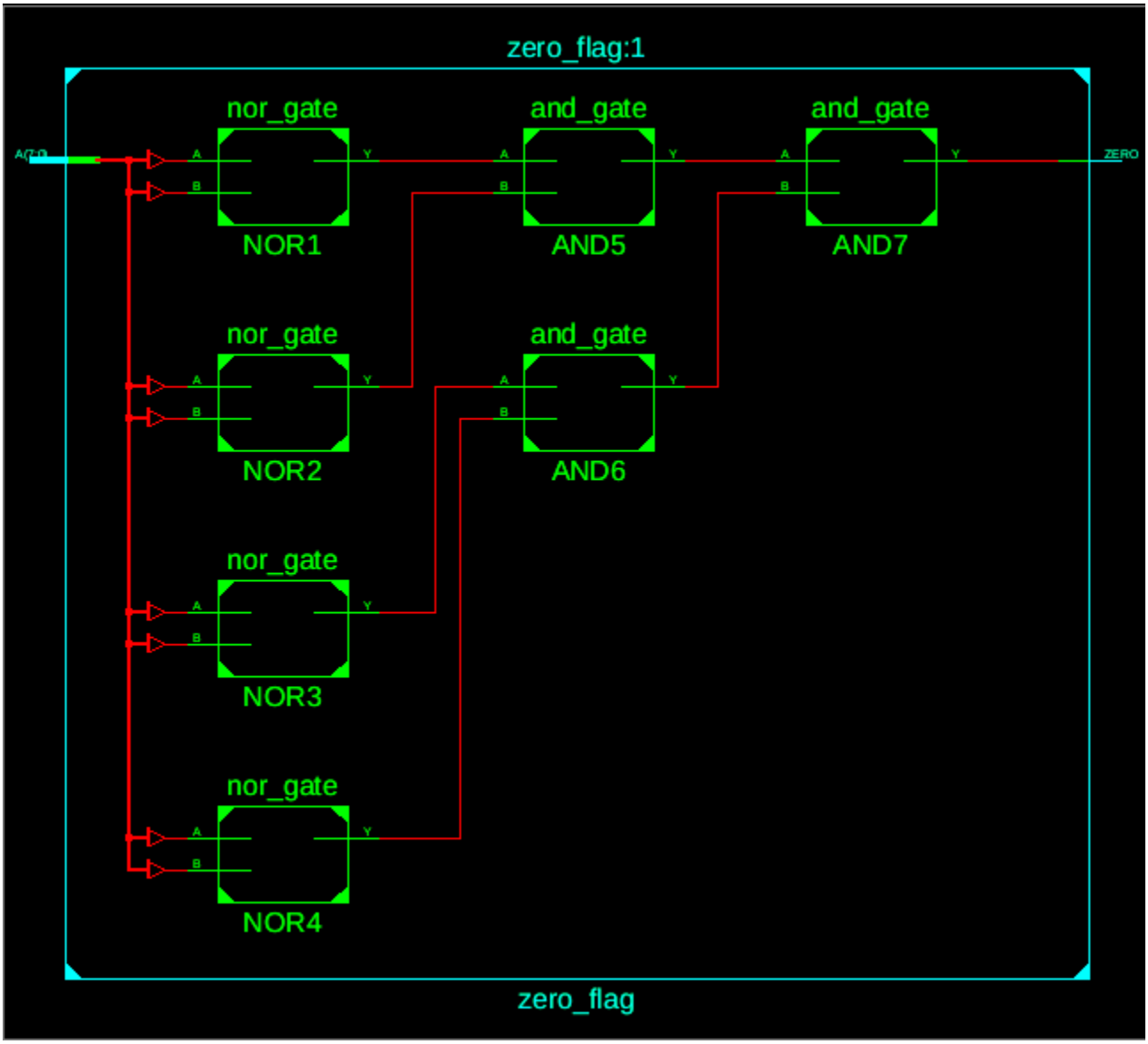


Figure 4: Block symbol of zero_flag

VHDL code: zero_flag.vhd

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;


entity zero_flag is

    Port (

                    A           :   in
STD_LOGIC_VECTOR(7 downto 0);

          ZERO   : out STD_LOGIC

    );
```

```
end zero_flag;


architecture   Structural_NOR   of
zero_flag is

    component nor_gate

        Port (A, B : in STD_LOGIC;
Y : out STD_LOGIC);

    end component;

    component and_gate

        Port (A, B : in STD_LOGIC;
Y : out STD_LOGIC);

    end component;

       signal  G1,  G2,  G3,  G4  :
STD_LOGIC;

      signal  G5,  G6             :
STD_LOGIC;

begin

    NOR1: nor_gate port map (A =>
A(0), B => A(1), Y => G1);

    NOR2: nor_gate port map (A =>
A(2), B => A(3), Y => G2);

    NOR3: nor_gate port map (A =>
A(4), B => A(5), Y => G3);

    NOR4: nor_gate port map (A =>
A(6), B => A(7), Y => G4);

    AND5: and_gate port map (A =>
G1, B => G2, Y => G5);

    AND6: and_gate port map (A =>
G3, B => G4, Y => G6);

    AND7: and_gate port map (A =>
G5, B => G6, Y => ZERO);

end Structural_NOR;
```

Testbench stimulus: zero_flag_tb.vhd (excerpt)

```
STIMULUS: PROCESS
BEGIN
    -- Test 1: All bits are zero
    A <= x"00";
    WAIT FOR 10 ns;
    ASSERT ZERO = '1' REPORT "Test 1 FAILED: Expected ZERO = 1" SEVERITY
ERROR;
    -- Test 2: Input is 1
    A <= x"01";
    WAIT FOR 10 ns;
    ASSERT ZERO = '0' REPORT "Test 2 FAILED: Expected ZERO = 0" SEVERITY
ERROR;
    -- Test 3: Input is FF
    A <= x"FF";
    WAIT FOR 10 ns;
    ASSERT ZERO = '0' REPORT "Test 3 FAILED: Expected ZERO = 0" SEVERITY
ERROR;
```

```
    -- Test 4: Input is 80
    A <= x"80";
    WAIT FOR 10 ns;
    ASSERT ZERO = '0' REPORT "Test 4 FAILED: Expected ZERO = 0" SEVERITY
ERROR;
    -- Test 5: Input is 0 again
    A <= x"00";
    WAIT FOR 10 ns;
    ASSERT ZERO = '1' REPORT "Test 5 FAILED: Expected ZERO = 1" SEVERITY
ERROR;
    REPORT "All zero flag tests completed." SEVERITY NOTE;
    WAIT;
END PROCESS;
```

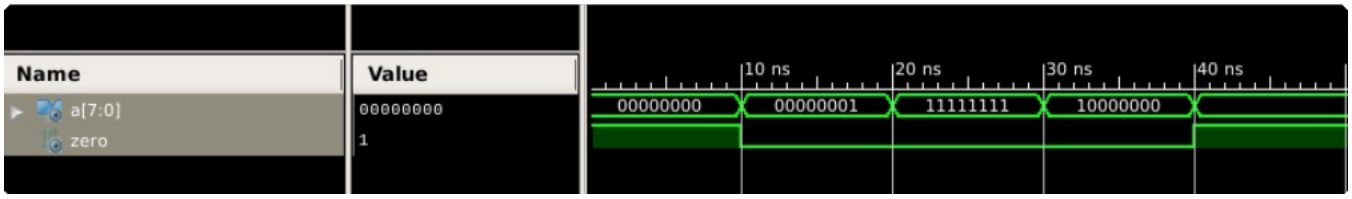


Figure 5: Simulation waveform of zero_flag

ZERO is 1 for A = 0x00 and drops to 0 for A = 0x01, 0xFF and 0x80. It returns to 1 when A = 0x00 again.

4.3 Accumulator (accumulator)

The accumulator is the main working register and the CPU output Q. It is one register_8bit: it loads D on every rising clock edge, and a synchronous, active-high RESET clears it to 0x00. The choice of what to load (operand, ALU result or the old value for a hold) is made by the ACC next-value logic in cpu_top, so the accumulator itself only stores.

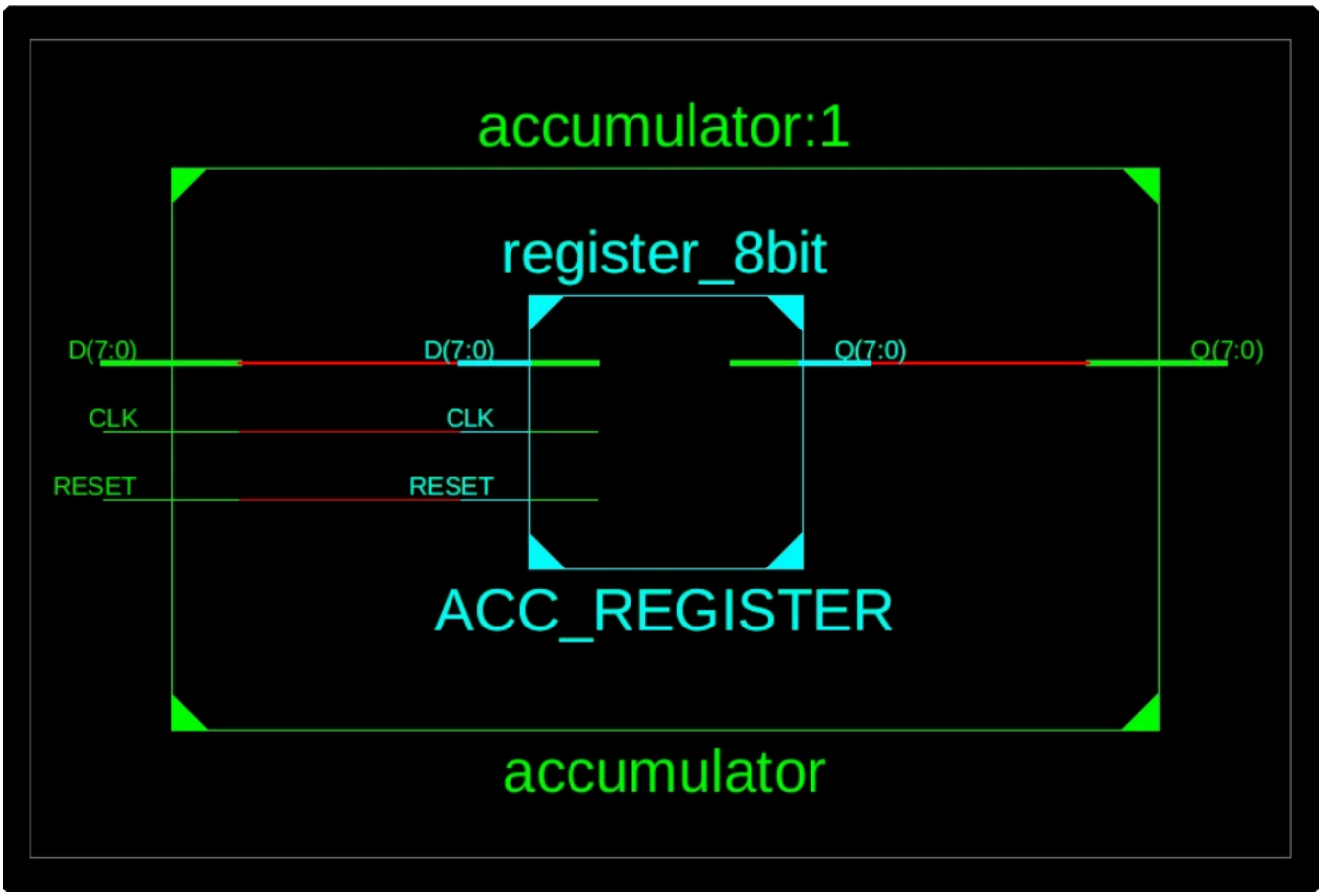


Figure 6: Block symbol of accumulator

VHDL code: accumulator.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity accumulator is
    Port (
        D   : in     STD_LOGIC_VECTOR(7 downto 0);
        CLK : in     STD_LOGIC;
        RESET : in   STD_LOGIC;
        Q   : out STD_LOGIC_VECTOR(7 downto 0)
    );
end accumulator;
architecture Structural of accumulator is
begin
    ACC_REGISTER : entity work.register_8bit
        port map (
            D     => D,
            CLK   => CLK,
```

```
            RESET => RESET,
            Q   => Q
        );
end Structural;
```

Testbench stimulus: accumulator_8bit_tb.vhd (excerpt)

```
STIMULUS: PROCESS
BEGIN
    -- Test 1: Reset
    RESET <= '1';
    D <= x"00";
    WAIT FOR 25
    ns;
    ASSERT Q = x"00" REPORT "Test 1 FAILED: Reset" SEVERITY ERROR;
    -- Test 2: Add 5
    RESET <= '0';
    D <= x"05";
    WAIT FOR 20 ns;
    ASSERT Q = x"05" REPORT "Test 2 FAILED: Expected 5" SEVERITY ERROR;
    -- Test 3: Add 10; total = 15
    D <= x"0A";
    WAIT FOR 20 ns;
    ASSERT Q = x"0F" REPORT "Test 3 FAILED: Expected 15" SEVERITY ERROR;
    -- Test 4: Add 20; total = 35
    D <= x"14";
    WAIT FOR 20 ns;
    ASSERT Q = x"23" REPORT "Test 4 FAILED: Expected 35" SEVERITY ERROR;
    -- Test 5: Add 255; 35 + 255 = 290, 8-bit result = 34
    D <= x"FF";
    WAIT FOR 20 ns;
    ASSERT Q = x"22" REPORT "Test 5 FAILED: Expected 34" SEVERITY ERROR;
    -- Test 6: Reset again
    RESET <= '1';
    D <= x"00";
    WAIT FOR 20
    ns;
    ASSERT Q = x"00" REPORT "Test 6 FAILED: Reset" SEVERITY ERROR;
    REPORT "All accumulator tests completed." SEVERITY NOTE;
    WAIT;
END PROCESS;
```

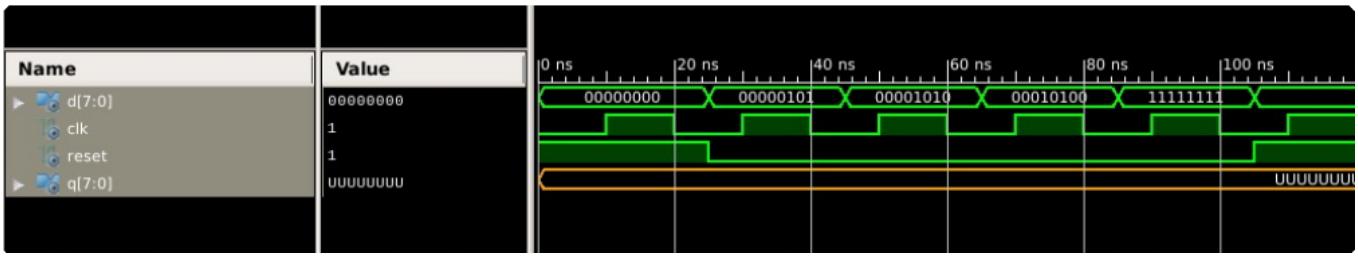


Figure 7: Simulation waveform of accumulator_8bit_tb

The stimulus applies RESET, then D = 0x05, 0x0A, 0x14 and 0xFF on successive clock edges, then RESET again. Q shows UUUUUUUU for the whole run. This testbench was written for the first version of the block, accumulator_8bit, which added D to its stored value (hence the expected 0x05, 0x0F, 0x23, 0x22). When the accumulator was redesigned as a plain register and renamed accumulator, the testbench still instantiated accumulator_8bit. ISim leaves an unbound component's outputs undefined, so Q stays U. For the redesigned accumulator the expected Q is 0x00, 0x05, 0x0A, 0x14, 0xFF and 0x00 again. The redesigned accumulator is verified inside cpu_top (Section 5), where ACC takes every expected value of the Set 3 program.

4.4 Program countcr (2rogram_countcr)

The program counter is the low nibble of an 8-bit register that holds the address of the current instruction. A full_adder_8bit adds 1 to the stored value, and the sum is loaded on every rising edge. A synchronous reset sets PC to 0. Set 3 has no jump instruction, so the PC needs no load input; the LOAD and D ports are left commented out in the entity.

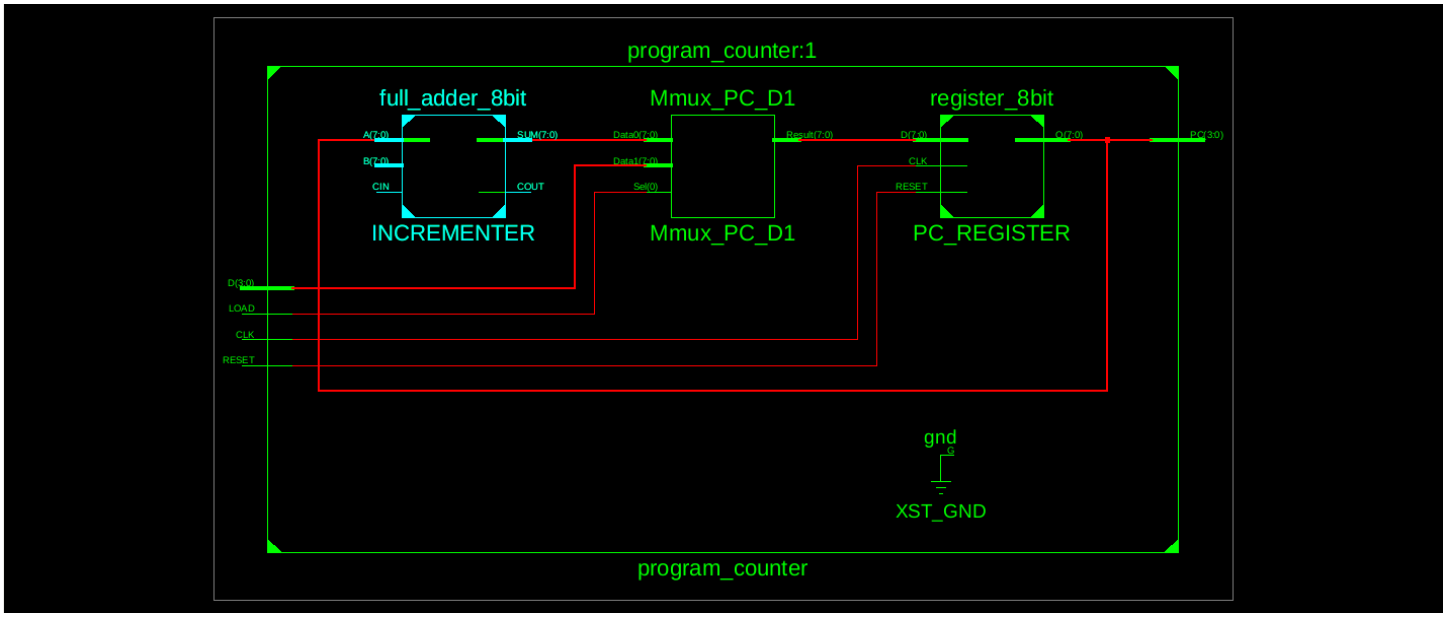


Figure 8: Block symbol of program_counter

VHDL code: program_counter.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity program_counter is
    Port (
        CLK   : in  STD_LOGIC;
        RESET : in  STD_LOGIC;
        LOAD  : in  STD_LOGIC := '0';
        D     : in  STD_LOGIC_VECTOR(3 downto 0) := "0000";
        PC    : out STD_LOGIC_VECTOR(3 downto 0)
    );
end program_counter;
architecture Structural of program_counter is

    component full_adder_8bit
        Port (
            A    : in  STD_LOGIC_VECTOR(7 downto 0);
            B    : in  STD_LOGIC_VECTOR(7 downto 0);
            CIN  : in  STD_LOGIC;
            SUM  : out STD_LOGIC_VECTOR(7 downto 0);
            COUT : out STD_LOGIC
        );
    end component;
    component register_8bit
        Port (
            D     : in  STD_LOGIC_VECTOR(7 downto 0);
            CLK   : in  STD_LOGIC;
            RESET : in  STD_LOGIC;
            Q     : out STD_LOGIC_VECTOR(7 downto 0)
        );
    end component;
    signal PC_CURRENT : STD_LOGIC_VECTOR(7 downto 0);
    signal PC_NEXT    : STD_LOGIC_VECTOR(7 downto 0);
    signal PC_D       : STD_LOGIC_VECTOR(7 downto 0);  -- NEW: register input (after
load mux)
    signal CARRY_OUT  : STD_LOGIC;
begin
    INCREMENTER: full_adder_8bit
        port map (
            A    => PC_CURRENT,
            B    => "00000001",
            CIN  => '0',
            SUM  => PC_NEXT,
            COUT => CARRY_OUT
        );
    -- NEW: choose between loading D and incrementing
    PC_D <= "0000" & D when LOAD = '1' else PC_NEXT;

    PC_REGISTER: register_8bit
        port map (
            D     => PC_D,          -- was PC_NEXT
            CLK   => CLK,
            RESET => RESET,
            Q     => PC_CURRENT
```

```
        );

    PC <= PC_CURRENT(3 downto 0);

end Structural;
```

Testbench stimulus: program_counter_tb.vhd (excerpt)

```
stim_proc : process
begin
    -- Apply synchronous reset for one rising edge
    RESET <= '1';
    wait for CLK_PERIOD; wait for 1 ns;
    assert PC = "0000" report "FAIL: PC after reset" severity error;
    -- Release reset: PC must count 1, 2, 3, ... 15, then wrap to 0
    RESET <= '0';
    for i in 1 to 16 loop
        wait for CLK_PERIOD;
        assert PC = std_logic_vector(to_unsigned(i mod 16, 4))
            report "FAIL: PC count at step " G integer'image(i) severity
error;
    end loop;
    -- Count a few more, then reset in the middle of counting
    wait for CLK_PERIOD;
    wait for CLK_PERIOD;
    wait for CLK_PERIOD;
    assert PC = "0011" report "FAIL: PC should be 3 before mid-run reset"
severity error;
    RESET <= '1';
    wait for CLK_PERIOD;
    assert PC = "0000" report "FAIL: PC after mid-run reset" severity error;
    -- Reset still held: PC must stay at 0
    wait for CLK_PERIOD;
    assert PC = "0000" report "FAIL: PC must stay 0 while RESET = 1" severity
error;
    -- Release again: counting restarts from 1
```

```
    RESET <= '0';
    wait for CLK_PERIOD;
    assert PC = "0001" report "FAIL: PC after reset release" severity error;
    report "program_counter testbench completed." severity note;
    wait;
end process;
```

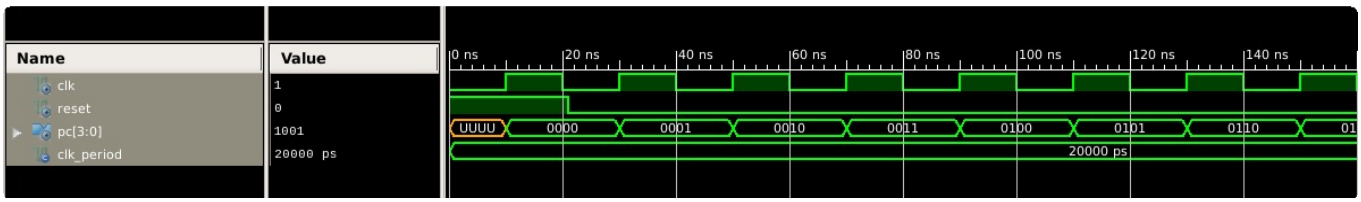


Figure 9: Simulation waveform of program_counter

PC is UUUU until the first rising edge at 10 ns, because the reset is synchronous. With RESET = 1 at that edge PC becomes 0000. After RESET is released PC counts 0001, 0010, 0011 and so on, one step per rising edge. The testbench also checks the wrap from 15 back to 0 and a reset in the middle of counting.

4.5 Program ROM (2rogram_rom)

The ROM is a 16 x 12-bit table written as a VHDL case statement. It is read combinationally with PC as the address (no clock). It holds the seven Set 3 instructions of Table 3, and every other address returns 0x000 (NOP). XST implements the case statement as a small read-only memory (Mram_INSTRUCTION1 with its write enable tied to ground).

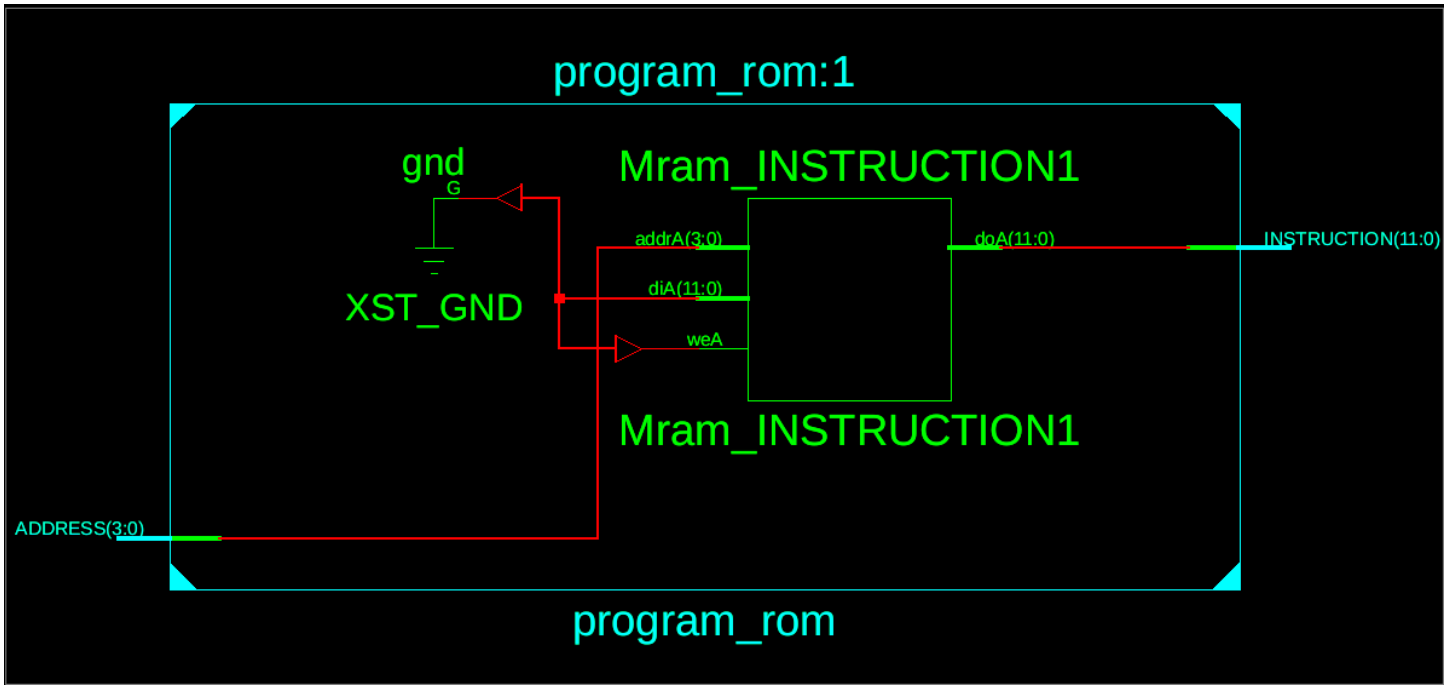


Figure 10: Block symbol of program_rom

VHDL code: program_rom.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity program_rom is
    Port (
        ADDRESS     : in  STD_LOGIC_VECTOR(3 downto 0);
        INSTRUCTION : out STD_LOGIC_VECTOR(11 downto 0)
    );
end program_rom;
architecture Array_ROM of program_rom is
    type rom_type is array (0 to 15) of STD_LOGIC_VECTOR(11 downto 0);
    constant ROM : rom_type := (
        0      => x"180",  -- LDI 0x80
        1      => x"900",  -- SHR
        2      => x"310",  -- SUB 0x10
        3      => x"20F",  -- ADD 0x0F
        4      => x"63F",  -- XOR 0x3F
        5      => x"207",  -- ADD 0x07
        6      => x"900",  -- SHR
        others => x"000"   -- NOP
    );
begin
    INSTRUCTION <= ROM(to_integer(unsigned(ADDRESS)));
end Array_ROM;
```

Testbench stimulus: program_rom_tb.vhd (excerpt)

```
stim_proc : process
begin
    ADDRESS <= "0000"; wait for 10 ns;
    assert INSTRUCTION = x"180" report "FAIL: ROM[0] LDI 80" severity error;
    ADDRESS <= "0001"; wait for 10 ns;
    assert INSTRUCTION = x"900" report "FAIL: ROM[1] SHR" severity error;
    ADDRESS <= "0010"; wait for 10 ns;
    assert INSTRUCTION = x"310" report "FAIL: ROM[2] SUB 10" severity error;
    ADDRESS <= "0011"; wait for 10 ns;
    assert INSTRUCTION = x"20F" report "FAIL: ROM[3] ADD 0F" severity error;
    ADDRESS <= "0100"; wait for 10 ns;
    assert INSTRUCTION = x"63F" report "FAIL: ROM[4] XOR 3F" severity error;
    ADDRESS <= "0101"; wait for 10 ns;
```

```
    assert INSTRUCTION = x"207" report "FAIL: ROM[5] ADD 07" severity error;
    ADDRESS <= "0110"; wait for 10 ns;
    assert INSTRUCTION = x"900" report "FAIL: ROM[6] SHR" severity error;
    -- Unused locations must be NOP (0x000)
    ADDRESS <= "0111"; wait for 10 ns;
    assert INSTRUCTION = x"000" report "FAIL: ROM[7] not NOP" severity error;
    ADDRESS <= "1000"; wait for 10 ns;
    assert INSTRUCTION = x"000" report "FAIL: ROM[8] not NOP" severity error;
    ADDRESS <= "1111"; wait for 10 ns;
    assert INSTRUCTION = x"000" report "FAIL: ROM[15] not NOP" severity
error;
    report "program_rom testbench completed." severity note;
    wait;
end process;
```

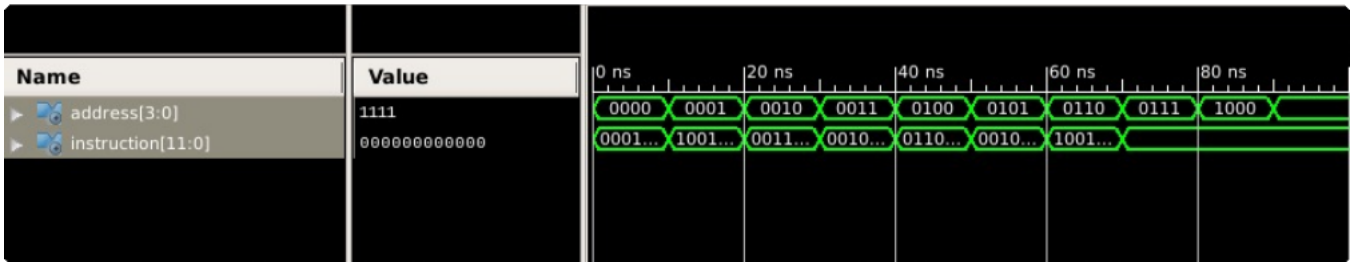


Figure 11: Simulation waveform of program_rom

Stepping the address from 0 to 6 gives 0x180, 0x900, 0x310, 0x20F, 0x63F, 0x207 and 0x900. Addresses 7, 8 and 15 return 0x000 (NOP). The output changes as soon as the address changes, which confirms the combinational read.

4.6 Instruction dccodcr (instruction_dccodcr)

The decoder splits the 12-bit instruction into OPCODE (bits 11:8) and OPERAND (bits 7:0). It is pure wiring with no logic, so it adds no delay. The opcode then goes to the control unit and the operand to the ALU and the ACC next-value logic.

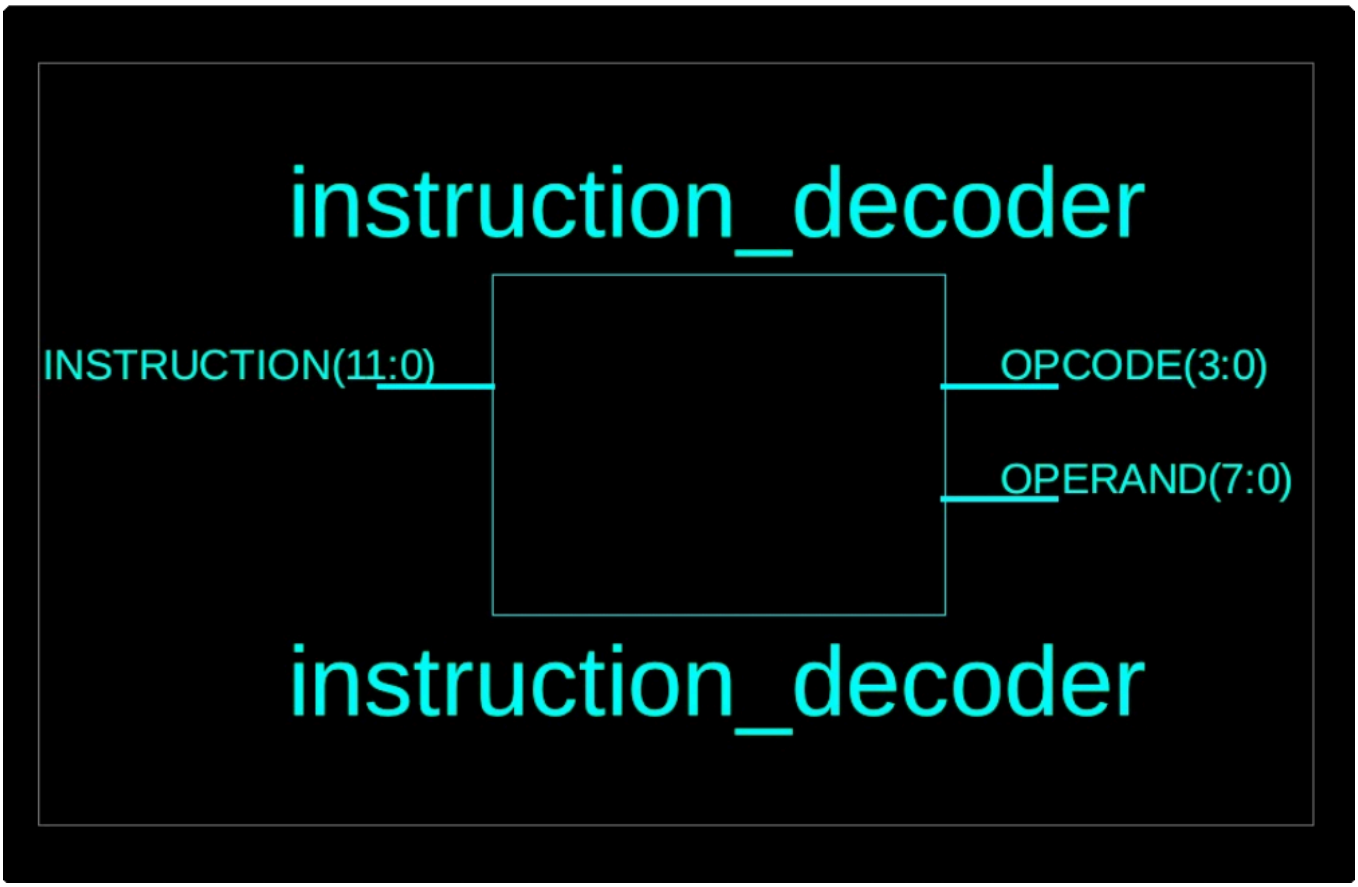


Figure 12: Block symbol of instruction_decoder

VHDL code: instruction_decoder.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity instruction_decoder is
    Port (
        INSTRUCTION : in STD_LOGIC_VECTOR(11 downto 0);
        OPCODE  : out STD_LOGIC_VECTOR(3 downto 0);
        OPERAND : out STD_LOGIC_VECTOR(7 downto 0)
    );
end instruction_decoder;
architecture Behavioral of instruction_decoder is
begin
    OPCODE <= INSTRUCTION(11 downto 8);
    OPERAND <= INSTRUCTION(7 downto 0);
end Behavioral;
```

Testbench stimulus: instruction_decoder_tb.vhd (excerpt)

```
stim_proc : process
begin
    INSTRUCTION <= x"180"; wait for 10 ns;   -- LDI 80
    assert OPCODE = "0001" and OPERAND = x"80" report "FAIL: LDI 80" severity
error;
    INSTRUCTION <= x"20F"; wait for 10 ns;   -- ADD 0F
    assert OPCODE = "0010" and OPERAND = x"0F" report "FAIL: ADD 0F" severity
error;
    INSTRUCTION <= x"310"; wait for 10 ns;   -- SUB 10
    assert OPCODE = "0011" and OPERAND = x"10" report "FAIL: SUB 10" severity
error;
    INSTRUCTION <= x"63F"; wait for 10 ns;   -- XOR 3F
    assert OPCODE = "0110" and OPERAND = x"3F" report "FAIL: XOR 3F" severity
error;
    INSTRUCTION <= x"900"; wait for 10 ns;   -- SHR
    assert OPCODE = "1001" and OPERAND = x"00" report "FAIL: SHR" severity
error;
    INSTRUCTION <= x"000"; wait for 10 ns;   -- NOP
    assert OPCODE = "0000" and OPERAND = x"00" report "FAIL: NOP" severity
error;
    report "instruction_decoder testbench completed." severity note;
    wait;
end process;
```

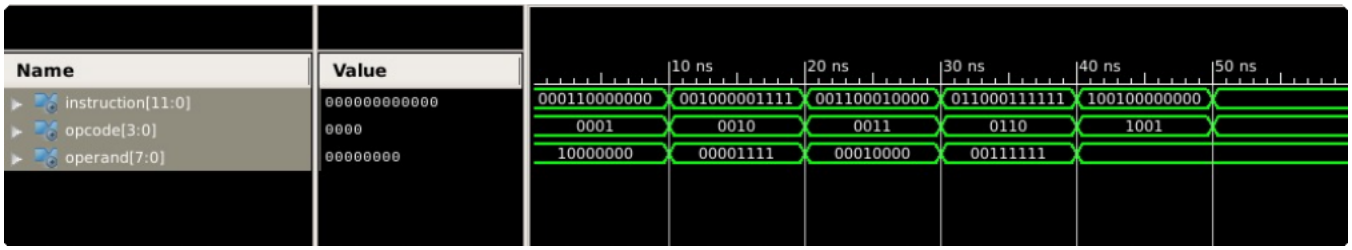


Figure 13: Simulation waveform of instruction_decoder

0x180 gives OPCODE 0001 and OPERAND 0x80, 0x20F gives 0010 and 0x0F, 0x310 gives 0011 and 0x10, 0x63F gives 0110 and 0x3F, and 0x900 gives 1001 and 0x00. For 0x000 both fields are zero (NOP).

4.7 Control unit (control_unit)

The control unit is a combinational block that turns the 4-bit opcode into the three datapath controls. LOAD_EN = 1 only for LDI. ACC_WR = 1 for every instruction that writes the accumulator (LDI, ADD, SUB, XOR, SHR). ALU_OP selects the ALU function: 00 ADD,

01 XOR, 10 SUB, 11 SHR. Every other opcode leaves all outputs at 0, which is a NOP. Being combinational, the controls are ready in the same cycle, so each instruction completes in one clock. XST implements the case statement as a 16-entry lookup table (Mram_n00211).

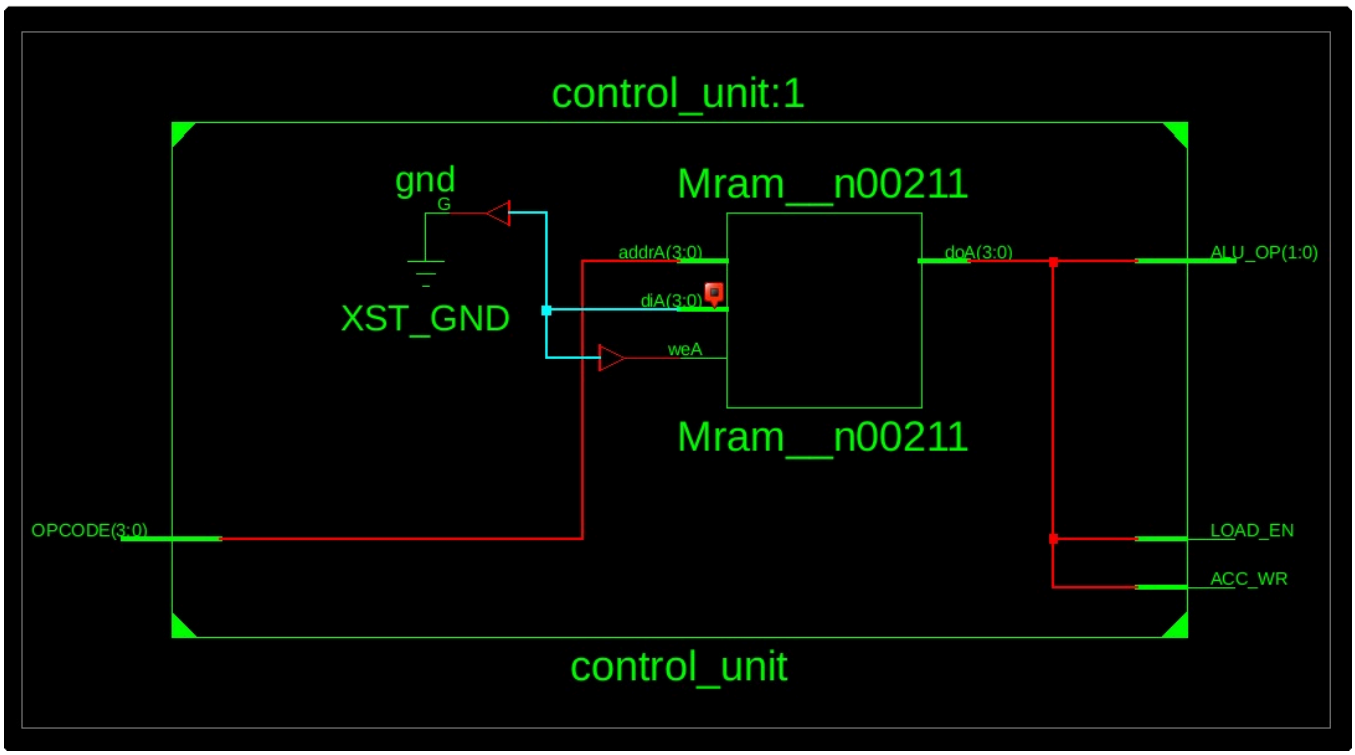


Figure 14: Block symbol of control_unit

VHDL code: control_unit.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity control_unit is
    Port (
        OPCODE   : in  STD_LOGIC_VECTOR(3 downto 0);
        LOAD_EN  : out STD_LOGIC;                      -- LDI
        ALU_OP   : out STD_LOGIC_VECTOR(1 downto 0);   -- 00 ADD, 01 XOR, 10
SUB, 11 SHR
        ACC_WR   : out STD_LOGIC                       -- write ACC this
cycle
    );
end control_unit;
architecture Behavioral of control_unit is
begin
    process(OPCODE)
    begin
```

```
        LOAD_EN <= '0';
        ALU_OP <= "00";
        ACC_WR <= '0';
        case OPCODE is
            when "0001" => LOAD_EN <= '1'; ACC_WR <= '1';   -- LDI
            when "0010" => ALU_OP <= "00"; ACC_WR <= '1';   -- ADD
            when "0011" => ALU_OP <= "10"; ACC_WR <= '1';   -- SUB
            when "0110" => ALU_OP <= "01"; ACC_WR <= '1';   -- XOR
            when "1001" => ALU_OP <= "11"; ACC_WR <= '1';   -- SHR
            when others => null;                            -- NOP

        end case;
    end process;
end Behavioral;
```

Testbench stimulus: control_unit_tb.vhd (excerpt)

```
stim_proc : process
begin
    -- NOP (0000)
    OPCODE <= "0000"; wait for 10 ns;
    assert LOAD_EN = '0' and ACC_WR = '0' report "FAIL: NOP" severity error;
    -- LDI (0001)
    OPCODE <= "0001"; wait for 10 ns;
    assert LOAD_EN = '1' and ACC_WR = '1' report "FAIL: LDI" severity error;
    -- ADD (0010)
    OPCODE <= "0010"; wait for 10 ns;
    assert LOAD_EN = '0' and ACC_WR = '1' and ALU_OP = "00" report "FAIL:
ADD" severity error;
    -- SUB (0011)
    OPCODE <= "0011"; wait for 10 ns;
    assert LOAD_EN = '0' and ACC_WR = '1' and ALU_OP = "10" report "FAIL:
SUB" severity error;
    -- XOR (0110)
    OPCODE <= "0110"; wait for 10 ns;
    assert LOAD_EN = '0' and ACC_WR = '1' and ALU_OP = "01" report "FAIL:
XOR" severity error;
    -- SHR (1001)
    OPCODE <= "1001"; wait for 10 ns;
    assert LOAD_EN = '0' and ACC_WR = '1' and ALU_OP = "11" report "FAIL:
SHR" severity error;
    -- Opcodes not in Set 3 must behave as NOP
    OPCODE <= "0100"; wait for 10 ns;
    assert LOAD_EN = '0' and ACC_WR = '0' report "FAIL: AND should be NOP"
```

```
severity error;
    OPCODE <= "0101"; wait for 10 ns;
    assert LOAD_EN = '0' and ACC_WR = '0' report "FAIL: OR should be NOP"
severity error;
    OPCODE <= "0111"; wait for 10 ns;
    assert LOAD_EN = '0' and ACC_WR = '0' report "FAIL: NOT should be NOP"
severity error;
    OPCODE <= "1000"; wait for 10 ns;
    assert LOAD_EN = '0' and ACC_WR = '0' report "FAIL: SHL should be NOP"
severity error;
    OPCODE <= "1010"; wait for 10 ns;
    assert LOAD_EN = '0' and ACC_WR = '0' report "FAIL: STA should be NOP"
severity error;
    OPCODE <= "1011"; wait for 10 ns;
    assert LOAD_EN = '0' and ACC_WR = '0' report "FAIL: LDR should be NOP"
severity error;
    OPCODE <= "1100"; wait for 10 ns;
    assert LOAD_EN = '0' and ACC_WR = '0' report "FAIL: JZ should be NOP"
severity error;
    report "control_unit testbench completed." severity note;
    wait;
end process;
```

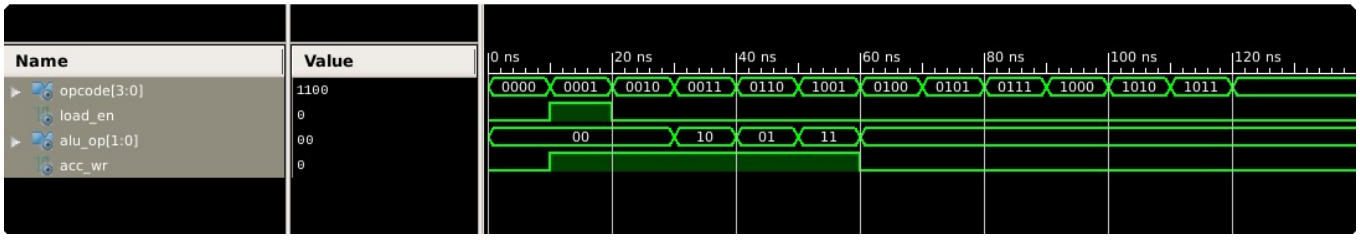


Figure 15: Simulation waveform of control_unit

Opcode 0000 produces no control signal. 0001 (LDI) raises LOAD_EN and ACC_WR. 0010 (ADD), 0011 (SUB), 0110 (XOR) and 1001 (SHR) raise ACC_WR with ALU_OP = 00, 10, 01 and
11. The opcodes outside Set 3 (0100, 0101, 0111, 1000, 1010, 1011, 1100) leave LOAD_EN and ACC_WR at 0, so they act as NOP.

4.8 To2-lcvcl CPU (c2u_to2)

cpu_top connects all blocks as shown in Figure 1. One conditional assignment implements the ACC next-value selection: the operand when LOAD_EN = 1 (LDI), the ALU result when ACC_WR = 1 (ADD, SUB, XOR, SHR), otherwise the current ACC value (NOP). The program

counter increments on every edge, and RESET goes to both the accumulator and the program counter. The outputs are Q = ACC, Z and PC.

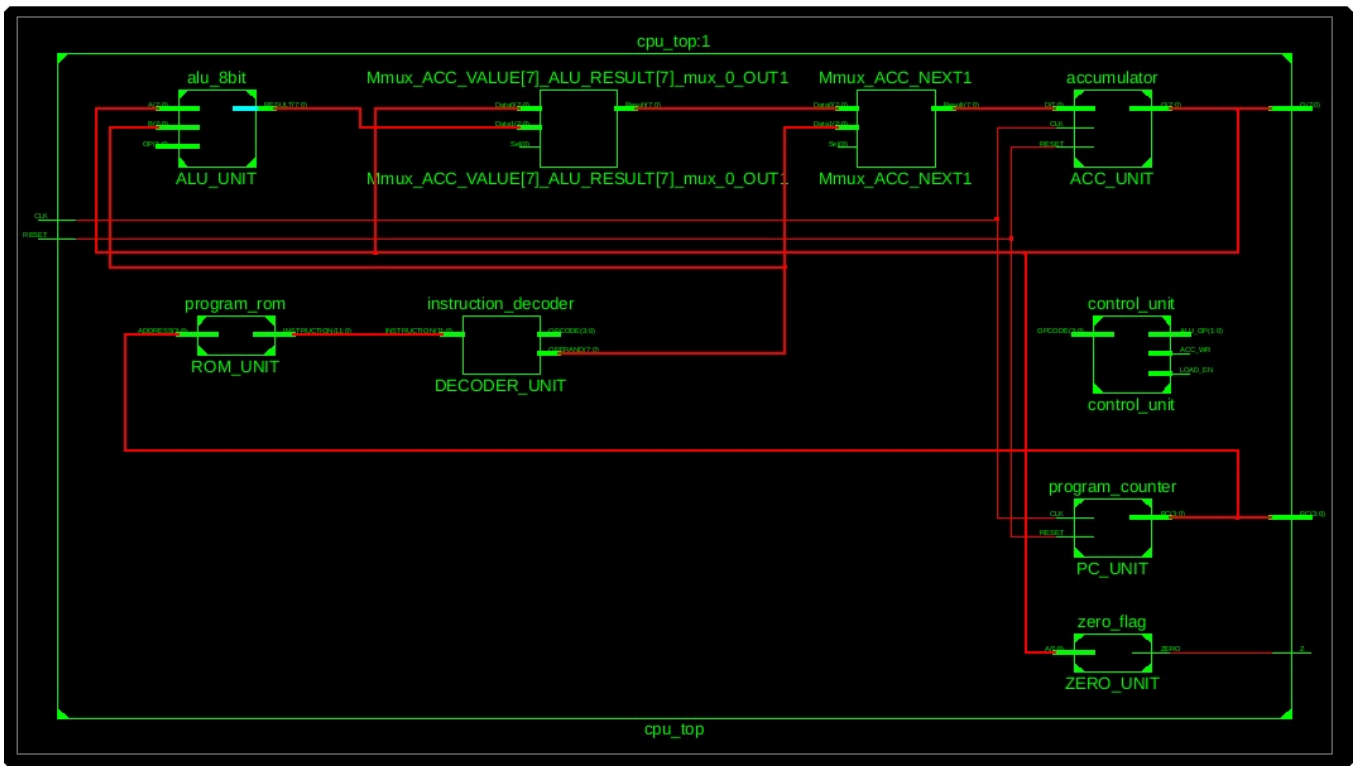


Figure 16: Block symbol of cpu_top

VHDL code: cpu_top.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity cpu_top is
    Port (
        CLK   : in STD_LOGIC;
        RESET : in STD_LOGIC;
        Q     : out STD_LOGIC_VECTOR(7 downto
        0); Z : out STD_LOGIC;
        PC    : out STD_LOGIC_VECTOR(3 downto 0)
    );
end cpu_top;
architecture Structural of cpu_top is
    signal PC_VALUE    : STD_LOGIC_VECTOR(3 downto 0);
    signal INSTRUCTION : STD_LOGIC_VECTOR(11 downto 0);
    signal OPCODE      : STD_LOGIC_VECTOR(3 downto 0);
    signal OPERAND     : STD_LOGIC_VECTOR(7 downto 0);
    signal LOAD_EN     : STD_LOGIC;
```

```
    signal  ALU_OP    :  STD_LOGIC_VECTOR(1  downto  0);
    signal ACC_WR   : STD_LOGIC;
    signal  ALU_RESULT  :  STD_LOGIC_VECTOR(7 downto 0);
    signal  ACC_NEXT  :  STD_LOGIC_VECTOR(7  downto  0);
    signal  ACC_VALUE  :  STD_LOGIC_VECTOR(7  downto  0);
    signal Z_VALUE  : STD_LOGIC;
begin
    ROM_UNIT : entity work.program_rom
        port map (ADDRESS => PC_VALUE, INSTRUCTION => INSTRUCTION);

    DECODER_UNIT : entity work.instruction_decoder
        port map (INSTRUCTION => INSTRUCTION, OPCODE => OPCODE, OPERAND =>
OPERAND);

    CONTROL_UNIT : entity work.control_unit
        port map (OPCODE => OPCODE, LOAD_EN => LOAD_EN, ALU_OP => ALU_OP,
ACC_WR => ACC_WR);

    ALU_UNIT : entity work.alu_8bit
        port map (A => ACC_VALUE, B => OPERAND, OP => ALU_OP, RESULT =>
ALU_RESULT);

    -- LDI -> operand, ADD/SUB/XOR/SHR -> ALU result, NOP -> hold
    ACC_NEXT <= OPERAND    when LOAD_EN = '1' else
                ALU_RESULT when ACC_WR = '1' else
                ACC_VALUE;

    ACC_UNIT : entity work.accumulator
        port map (D => ACC_NEXT, CLK => CLK, RESET => RESET, Q => ACC_VALUE);

    ZERO_UNIT : entity work.zero_flag
        port map (A => ACC_VALUE, ZERO => Z_VALUE);

    PC_UNIT : entity work.program_counter
        port map (CLK => CLK, RESET => RESET, PC => PC_VALUE);

    Q <= ACC_VALUE;
    Z <= Z_VALUE;
    PC <= PC_VALUE;
end Structural;
```

5. To2-Lcvcl Tcstbcnch and Rcsults

The testbench generates a 20 ns clock, applies RESET for one rising edge, and then runs one clock cycle for each instruction of the Set 3 program. After every edge it checks Q (ACC), Z and PC with assert statements against the expected Appendix B values. The complete testbench code follows.

Testbench code: cpu_top_tb.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY cpu_top_tb IS
END cpu_top_tb;

ARCHITECTURE behavior OF cpu_top_tb IS
    COMPONENT cpu_top
        PORT (
            CLK   : IN std_logic;
            RESET : IN std_logic;
            Q     : OUT std_logic_vector(7 downto
            0); Z : OUT std_logic;
            PC    : OUT std_logic_vector(3 downto 0)
        );
    END COMPONENT;
    -- Inputs
    signal CLK   : std_logic := '0';
    signal RESET : std_logic := '0';
    -- Outputs
    signal Q : std_logic_vector(7 downto 0);
    signal Z : std_logic;
    signal PC : std_logic_vector(3 downto 0);
    constant CLK_PERIOD : time := 20 ns;
BEGIN
    -- Instantiate the CPU
    uut : cpu_top
        PORT MAP (
            CLK   => CLK,
            RESET => RESET,
            Q     => Q,
            Z     => Z,
            PC    => PC
```

```
        );
    -- Clock generation: 20 ns period
    clk_process : process
    begin
        CLK <= '0';
        wait for CLK_PERIOD / 2;
        CLK <= '1';
        wait for CLK_PERIOD / 2;
    end process;
    -- Set 3 test sequence (LDI, ADD, SHR, SUB, XOR)
    stim_proc : process
    begin
        RESET <= '1';
        wait for CLK_PERIOD; wait for 1 ns;
        assert Q = x"00" and Z = '1' and PC = "0000" report "FAIL: Reset
state" severity error;
        RESET <= '0';
        wait for CLK_PERIOD; wait for 1 ns;   -- LDI 0x80
        assert Q = x"80" and Z = '0' and PC = "0001" report "FAIL: LDI 80"
severity error;

        wait for CLK_PERIOD;                  -- SHR
        assert Q = x"40" and Z = '0' and PC = "0010" report "FAIL: SHR (addr
1)" severity error;
        wait for CLK_PERIOD;                  -- SUB 0x10
        assert Q = x"30" and Z = '0' and PC = "0011" report "FAIL: SUB 10"
severity error;

        wait for CLK_PERIOD;                  -- ADD 0x0F
        assert Q = x"3F" and Z = '0' and PC = "0100" report "FAIL: ADD 0F"
severity error;
        wait for CLK_PERIOD;                  -- XOR 0x3F
        assert Q = x"00" and Z = '1' and PC = "0101" report "FAIL: XOR 3F"
severity error;

        wait for CLK_PERIOD;                  -- ADD 0x07
        assert Q = x"07" and Z = '0' and PC = "0110" report "FAIL: ADD 07"
severity error;
        wait for CLK_PERIOD;                  -- SHR
        assert Q = x"03" and Z = '0' and PC = "0111" report "FAIL: SHR (addr
6)" severity error;
```

```
        report "Set 3 CPU testbench completed." severity note;
        wait;
    end process;
END behavior;
```

5.1 Simulation wavcfiorm

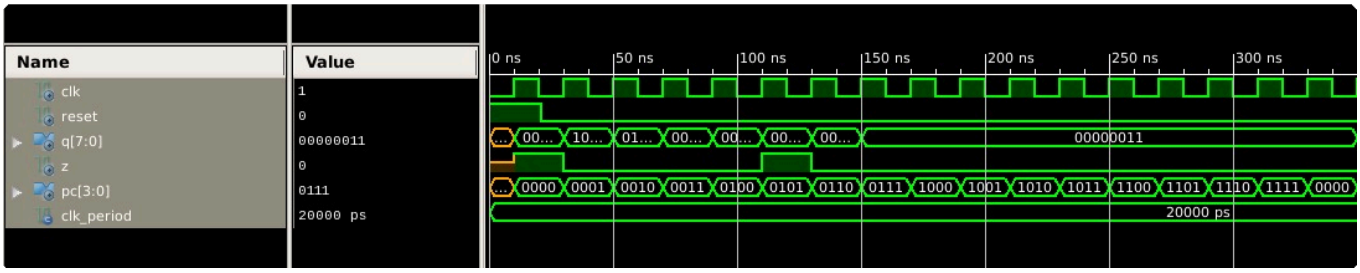


Figure 17: Waveform of cpu_top running the Set 3 program (CLK, RESET, Q = ACC, Z, PC)

RESET is high at the first rising edge (10 ns), so PC = 0x0, ACC = 0x00 and Z = 1. The program then runs one instruction per rising edge:

- **LDI 0x80** loads ACC = 0x80 and Z becomes 0.
- **SHR** shifts ACC right: 0x80 becomes 0x40.
- **SUB 0x10** gives 0x40 - 0x10 = 0x30.
- **ADD 0x0F** gives 0x30 + 0x0F = 0x3F.
- **XOR 0x3F** gives 0x3F XOR 0x3F = 0x00, so Z becomes 1 (the high pulse on z near 110 ns).
- **ADD 0x07** gives ACC = 0x07 and Z = 0.
- **SHR** gives 0x07 >> 1 = 0x03 and PC = 7.

After address 6 the ROM returns NOP, so ACC stays at 0x03 while PC keeps counting to 1111 and wraps to 0000 at about 330 ns.

5.2 Rcsult chcck against A22cndix B

Table 4: Simulated values after each rising edge compared with the Appendix B table for Set 3

Cycle	PC	Instruction executed	ACC (hex)	ACC (binary)	Z	Next PC	Status
0	-	RESET = 1	0x00	00000000	1	0	Pass
1	0	LDI 0x80	0x80	10000000	0	1	Pass
2	1	**SHR**	0x40	01000000	0	2	Pass

Cycle	PC	Instruction executed	ACC (hex)	ACC (binary)	Z	Next PC	Status
3	2	**SUB 0x10**	0x30	00110000	0	3	Pass
4	3	ADD 0x0F	0x3F	00111111	0	4	Pass
5	4	**XOR 0x3F**	0x00	00000000	1	5	Pass
6	5	ADD 0x07	0x07	00000111	0	6	Pass
7	6	**SHR**	0x03	00000011	0	7	Pass

All eight cycles match the expected ACC, Z and next-PC values of Appendix B (Set 3).

6. Challcngcs Faccd

- **Building everything from NAND gates.** The gate-level hierarchy is deep (NAND, gates, adders, latches, flip-flops, registers, ALU, CPU). Debugging was done bottom-up: the modules got their own testbenches and were checked before being used in a larger block.
- **Accumulator behaviour.** The first accumulator (accumulator_8bit) added its input to its stored value. That is correct only for ADD and wrong for LDI, SUB, XOR and SHR. It was redesigned as a plain register, and the choice of the next ACC value moved into cpu_top (operand, ALU result or hold).
- **Stale accumulator testbench.** After the redesign the entity was renamed accumulator, but accumulator_8bit_tb still instantiated accumulator_8bit and still expected the adding behaviour. ISim left the unbound component's output undefined, so Figure 7 shows Q = UUUUUUUU. The redesigned accumulator was verified through the top-level run instead (Section 5).
- **Four ALU operations.** Set 3 needs ADD, SUB, XOR and SHR, so the ALU select had to be 2 bits wide. SUB reuses the adder design with every bit of B inverted and carry-in = 1 (two's complement), and SHR is plain wiring with a 0 entering the MSB.
- **Program counter without jumps.** Set 3 has no JZ, so the PC only needs PC + 1. The LOAD and D ports of the program counter were left out (commented in the entity). Because the PC has no halt, after address 15 it wraps to 0 and the program starts again.
- **Flip-flops built from latches.** The register uses master-slave flip-flops made of NAND latches, with the synchronous reset done by a mux in front of them. This works in simulation, but registers are undefined (U) until the first clock edge, and XST maps

the latch loops onto LUTs instead of real flip-flops, so the synthesis report finds no clocked path.

- **Timing of the checks.** Z is derived directly from ACC, so it is 1 after reset and changes in the same cycle as ACC. The testbench samples the outputs 1 ns after each rising edge to avoid race conditions.

7. Conclusion

A single-cycle 8-bit accumulator CPU for Set 3 (LDI, ADD, **SHR**, **SUB**, **XOR**) was designed in VHDL using a modular, gate-level hierarchy built from NAND gates. The design contains separate entities for the PC, Program ROM, decoder, control unit, ALU, accumulator and zero flag, connected in a top-level cpu_top entity with a synchronous active-high reset and the outputs Q[7:0], Z and PC[3:0]. The Appendix B program was stored in the ROM and executed by the testbench: ACC, Z and PC match the expected values on every cycle, including the XOR that clears ACC and sets Z. The project showed how a complete processor can be built step by step from one primitive gate and verified at each level.

8. Submitted Files

- **VHDL entities:** nand_gate, not_gate, and_gate, or_gate, xor_gate, half_adder, full_adder, full_adder_8bit, mux_8bit, d_latch, d_flip_flop, register_8bit, alu_8bit, zero_flag, accumulator, program_counter, program_rom, instruction_decoder, control_unit, cpu_top (.vhd).
- **Testbenches:** nand_gate_tb, not_gate_tb, and_gate_tb, or_gate_tb, xor_gate_tb, half_adder_tb, full_adder_tb, full_adder_8bit_tb, mux_8bit_tb, register_8bit_tb, alu_8bit_tb, zero_flag_tb, accumulator_8bit_tb, program_counter_tb, program_rom_tb, instruction_decoder_tb, control_unit_tb and cpu_top_tb.vhd for the Set 3 program.

9. GitHub Link

https://github.com/IstiakNabil/Vlsi-Lab-Report/tree/main/cpu_design/cpu_design